RV32GC

Sliderule Decoder

Instruction Set "Sliderule" Decoder by Simone Shawn Cazzaniga (magiwanders@github | www.magiwanders.com | shawn@magiwanders.com)

Bitwise Breakdown — inst[31:0]																																Meaning						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Instruction	Mnemonic	Operands	Expansion	Pseudocode	Type	Ext
Quadrant 0 — Compressed Instructions (inst[1:0] = 00)																																						
[redacted]																**0**	**0**	**0**	imm									rd'		0	0	**Put SP + nzuimm in rd'**				**rd' = sp + nzuimm**	**CIW - Type** "Wide Immediate"	**'C'**
– 0 –																9:6										5:4		3	2	– 0 –		nzuimm				CIW zero-extended immediate		
[redacted]																**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	– any –				0	0	**RESERVED (nzuimm = 0) — 0x0000 is the defined illegal instruction**						
[redacted]																**0**	**0**	**0**	nzui...			nzuimm[9:6]			**2**	3		rd'		0	0	Add Immediate (×4) to SP i...	**c.addi4spn**	rd', sp, imm	addi rd', sp, nzuimm	rd' = sp + nzuimm		
[redacted]																**1**	**0**	**0**	imm			rs1'			imm			rd'		0	0	**Reserved**					**CL - Type** "Load"	**'C'**
[redacted]																**1**	**0**	**0**	– any –											0	0	**RESERVED**						
[redacted]																**0**	**1**	**0**	imm			rs1'			imm			rd'		0	0	**Load Word**				**rd' ← M[rs1' + uimm][31:0]**	**CL - Type** "Load"	**'C'**
– 0 –																									6		5:3		2	– 0 –		uimm				CL zero-extended immediate		
[redacted]																**0**	**1**	**0**	uimm[5:3]			rs1'			2	6		rd'		0	0	Load Word	**c.lw**	rd', imm(rs1')	lw rd', uimm(rs1')	rd' ← M[rs1' + uimm][31:0]		
[redacted]																**1**	**1**	**0**	imm			rs1'			imm			rs2'		0	0	**Store Word**				**rs2' → M[rs1' + uimm][31:0]**	**CS - Type** "Store"	**'C'**
– 0 –																									6		5:3		2	– 0 –		uimm				CS zero-extended immediate		
[redacted]																**1**	**1**	**0**	uimm[5:3]			rs1'			2	6		rs2'		0	0	Store Word	**c.sw**	rs2', imm(rs1')	sw rs2', uimm(rs1')	rs2' → M[rs1' + uimm][31:0]		
[redacted]																**0**	**0**	**1**	imm			rs1'			imm			rd'		0	0	**Load Double (FP)**				**rd' ← M[rs1' + uimm][63:0]**	**CL - Type** "Load"	**'C + D'**
– 0 –																								7:6			5:3			– 0 –		uimm				CL zero-extended immediate		
[redacted]																**0**	**0**	**1**	uimm[5:3]			rs1'			uim...			rd'		0	0	Load Double (FP)	**c.fld**	rd', imm(rs1')	fld rd', uimm(rs1')	rd' ← M[rs1' + uimm][63:0]		
[redacted]																**1**	**0**	**1**	imm			rs1'			imm			rs2'		0	0	**Store Double (FP)**				**rs2' → M[rs1' + uimm][63:0]**	**CS - Type** "Store"	**'C + D'**
– 0 –																								7:6			5:3			– 0 –		uimm				CS zero-extended immediate		
[redacted]																**1**	**0**	**1**	uimm[5:3]			rs1'			uim...			rs2'		0	0	Store Double (FP)	**c.fsd**	rs2', imm(rs1')	fsd rs2', uimm(rs1')	rs2' → M[rs1' + uimm][63:0]		
[redacted]																**0**	**1**	**1**	imm			rs1'			imm			rd'		0	0	**Load Float (FP)**				**rd' ← M[rs1' + uimm][31:0]**	**CL - Type** "Load"	**'C + F'**
– 0 –																									6		5:3		2	– 0 –		uimm				CL zero-extended immediate		
[redacted]																**0**	**1**	**1**	uimm[5:3]			rs1'			2	6		rd'		0	0	Load Float (FP)	**c.flw**	rd', imm(rs1')	flw rd', uimm(rs1')	rd' ← M[rs1' + uimm][31:0]		
[redacted]																**1**	**1**	**1**	imm			rs1'			imm			rs2'		0	0	**Store Float (FP)**				**rs2' → M[rs1' + uimm][31:0]**	**CS - Type** "Store"	**'C + F'**
– 0 –																									6		5:3		2	– 0 –		uimm				CS zero-extended immediate		
[redacted]																**1**	**1**	**1**	uimm[5:3]			rs1'			2	6		rs2'		0	0	Store Float (FP)	**c.fsw**	rs2', imm(rs1')	fsw rs2', uimm(rs1')	rs2' → M[rs1' + uimm][31:0]		
Quadrant 2 — Compressed Instructions (inst[1:0] = 10)																																						
[redacted]																**0**	**0**	**0**	i...			rd/rs1				imm				1	0	**Shift Left Logical Immediate**				**rd <<= shamt**	**CI - Type** "Immediate"	**'C'**
– 0 –																												4:0				shamt				CI zero-extended immediate		
[redacted]																**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	shamt[4:0]				1	0	*HINT — c.slli with rd = x0 (custom use) : PC += 2, no other effect*						
[redacted]																**0**	**0**	**0**	**0**	rd/rs1 ≠ 0					**0**	**0**	**0**	**0**	**0**	1	0	*HINT — c.slli with shamt = 0 (custom use) : PC += 2, no other effect*						
[redacted]																**0**	**0**	**0**	**0**	rd/rs1 ≠ 0					shamt[4:0]					1	0	Shift Left Logical Immediate	**c.slli**	rd, imm	slli rd, rd, shamt	rd <<= shamt		
[redacted]																**0**	**0**	**0**	**1**	– any –					– any –					1	0	**CUSTOM — c.slli with shamt[5] = 1: designated for custom extensions (RV32)**						
[redacted]																**funct4**				rd/rs1					rs2					1	0	**CR Control Transfer / Register Operations**				**(ra = PC + 2 ;) PC = rs1 | rd = rd …**	**CR - Type** "Register"	**'C'**
[redacted]																**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	1	0	**RESERVED (rs1 = 0)**						
[redacted]																**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs2 ≠ 0					1	0	*HINT — c.mv with rd = x0 (standard use) : PC += 2, no other effect*						
[redacted]																**1**	**0**	**0**	**0**	rs1 ≠ 0					**0**	**0**	**0**	**0**	**0**	1	0	Jump Register	**c.jr**	rs1	jalr x0, 0(rs1)	PC = rs1		
[redacted]																**1**	**0**	**0**	**0**	rd ≠ 0					rs2 ≠ 0					1	0	Move Register	**c.mv**	rd, rs2	add rd, x0, rs2	rd = rs2		
[redacted]																**1**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	1	0	Environment Break	**c.ebreak**	–	ebreak	control back to debug environment		
[redacted]																**1**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	rs2 ≠ 0					1	0	*HINT — c.add with rd = x0 (standard use; rs2 = x2–x5 are c.ntl.p1 / pall / s1 / all) : PC += 2, no other effect*						
[redacted]																**1**	**0**	**0**	**1**	rs1 ≠ 0					**0**	**0**	**0**	**0**	**0**	1	0	Jump and Link Register	**c.jalr**	rs1	jalr x1, 0(rs1)	ra = PC + 2 ; PC = rs1		
[redacted]																**1**	**0**	**0**	**1**	rd/rs1 ≠ 0					rs2 ≠ 0					1	0	Add	**c.add**	rd, rs2	add rd, rd, rs2	rd += rs2		
[redacted]																**0**	**1**	**0**	i...			rd/rs1				imm				1	0	**Load Word from SP**				**rd ← M[sp + uimm][31:0]**	**CI - Type** "Immediate"	**'C'**
– 0 –																								7:6		5		4:2	– 0 –		uimm				CI zero-extended immediate		
[redacted]																**0**	**1**	**0**	–	**0**	**0**	**0**	**0**	**0**	– any –					1	0	**RESERVED (rd = 0)**						
[redacted]																**0**	**1**	**0**	5	rd ≠ 0					uimm[4:2]		uim...		1	0	Load Word from SP	**c.lwsp**	rd, imm(sp)	lw rd, uimm(sp)	rd ← M[sp + uimm][31:0]		
[redacted]																**1**	**1**	**0**	imm						rs2					1	0	**Store Word to SP**				**rs2 → M[sp + uimm][31:0]**	**CSS - Type** "Stack-relative Store"	**'C'**
– 0 –																								7:6			5:2			– 0 –		uimm				CSS zero-extended immediate		
[redacted]																**1**	**1**	**0**	uimm[5:2]			uim...			rs2					1	0	Store Word to SP	**c.swsp**	rs2, imm(sp)	sw rs2, uimm(sp)	rs2 → M[sp + uimm][31:0]		
[redacted]																**0**	**0**	**1**	i...			rd/rs1				imm				1	0	**Load Double from SP (FP)**				**rd ← M[sp + uimm][63:0]**	**CI - Type** "Immediate"	**'C + D'**
– 0 –																										8:6		5	4:3	– 0 –		uimm				CI zero-extended immediate		
[redacted]																**0**	**0**	**1**	**5**	rd					uim...			uimm[8:6]		1	0	Load Double from SP (FP)	**c.fldsp**	rd, imm(sp)	fld rd, uimm(sp)	rd ← M[sp + uimm][63:0]		
[redacted]																**1**	**0**	**1**	imm						rs2					1	0	**Store Double to SP (FP)**				**rs2 → M[sp + uimm][63:0]**	**CSS - Type** "Stack-relative Store"	**'C + D'**
– 0 –																										8:6		5:3		– 0 –		uimm				CSS zero-extended immediate		
[redacted]																**1**	**0**	**1**	uimm[5:3]			uimm[8:6]			rs2					1	0	Store Double to SP (FP)	**c.fsdsp**	rs2, imm(sp)	fsd rs2, uimm(sp)	rs2 → M[sp + uimm][63:0]		
[redacted]																**0**	**1**	**1**	i...			rd/rs1				imm				1	0	**Load Float from SP (FP)**				**rd ← M[sp + uimm][31:0]**	**CI - Type** "Immediate"	**'C + F'**
– 0 –																								7:6		5	4:2		– 0 –		uimm				CI zero-extended immediate		
[redacted]																**0**	**1**	**1**	**5**	rd					uimm[4:2]		uim...		1	0	Load Float from SP (FP)	**c.flwsp**	rd, imm(sp)	flw rd, uimm(sp)	rd ← M[sp + uimm][31:0]		
[redacted]																**1**	**1**	**1**	imm						rs2					1	0	**Store Float to SP (FP)**				**rs2 → M[sp + uimm][31:0]**	**CSS - Type** "Stack-relative Store"	**'C + F'**
– 0 –																								7:6			5:2		– 0 –		uimm				CSS zero-extended immediate		
[redacted]																**1**	**1**	**1**	uimm[5:2]			uim...			rs2					1	0	Store Float to SP (FP)	**c.fswsp**	rs2, imm(sp)	fsw rs2, uimm(sp)	rs2 → M[sp + uimm][31:0]		
Quadrant 1 — Compressed Instructions (inst[1:0] = 01)																																						
[redacted]																**0**	**0**	**0**	i...			rd/rs1				imm				0	1	**Add Immediate / No Operation**				**rd += nzimm**	**CI - Type** "Immediate"	**'C'**
– 5 –																												5	4:0			nzimm				CI sign-extended immediate		
[redacted]																**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	0	1	No Operation	**c.nop**	–	addi x0, x0, 0	literally do nothing		
[redacted]																**0**	**0**	**0**	**0**	rd/rs1 ≠ 0					**0**	**0**	**0**	**0**	**0**	0	1	*HINT — c.addi with nzimm = 0 (standard use) : PC += 2, no other effect*						
[redacted]																**0**	**0**	**0**	5	**0**	**0**	**0**	**0**	**0**	nzimm[4:0]					0	1	*HINT — c.nop with nzimm ≠ 0 (standard use) : PC += 2, no other effect*						

Bitwise Breakdown — inst[31:0]																																Meaning						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Instruction	Mnemonic	Operands	Expansion	Pseudocode	Type	Ext
																0	0	0	5	rd/rs1 ≠ 0					nzimm[4:0]			0	1	Add Immediate	c.addi	rd, imm	addi rd, rd, nzimm	rd += nzimm		
																1	0	0	offset			rs1'			offset			0	1	**Shift Right / And Immediate**				**rd' = rd' (funct2) imm**	**CB - Type** "Branch / Arithmetic"	**'C'**
																1	0	0	0	0	0	rd'/rs1'	0	0	0	0	0	0	1	*HINT — c.srli with shamt = 0 (custom use) : PC += 2, no other effect*						
																1	0	0	0	0	0	rd'/rs1'	shamt[4:0]					0	1	Shift Right Logical Immediate	c.srli	rd', imm	srli rd', rd', shamt	rd' = rd' >> shamt (zero extends)		
– 0 –																											4:0			shamt				CB zero-extended immediate		
																1	0	0	0	0	1	rd'/rs1'	0	0	0	0	0	0	1	*HINT — c.srai with shamt = 0 (custom use) : PC += 2, no other effect*						
																1	0	0	0	0	1	rd'/rs1'	shamt[4:0]					0	1	Shift Right Arithmetic Imm…	c.srai	rd', imm	srai rd', rd', shamt	rd' = rd' >> shamt (msb extends)		
– 0 –																											4:0			shamt				CB zero-extended immediate		
																1	0	0	1	0	–	– any –	– any –					0	1	**CUSTOM — c.srli / c.srai with shamt[5] = 1: designated for custom extensions (RV32)**						
																1	0	0	5	1	0	rd'/rs1'	imm[4:0]					0	1	And Immediate	c.andi	rd', imm	andi rd', rd', imm	rd' &= imm		
– 5 –																										5	4:0			imm				CB sign-extended immediate		
																funct6						rd'/rs1'	funct2		rs2'			0	1	**CA Register Operations**				**rd' = rd' (funct2) rs2'**	**CA - Type** "Arithmetic"	**'C'**
																1	0	0	0	1	1	rd'/rs1'	0	0	rs2'			0	1	Subtract	c.sub	rd', rs2'	sub rd', rd', rs2'	rd' -= rs2'		
																1	0	0	0	1	1	rd'/rs1'	1	0	rs2'			0	1	Or	c.or	rd', rs2'	or rd', rd', rs2'	rd' |= rs2'		
																1	0	0	0	1	1	rd'/rs1'	0	1	rs2'			0	1	Xor	c.xor	rd', rs2'	xor rd', rd', rs2'	rd' ^= rs2'		
																1	0	0	0	1	1	rd'/rs1'	1	1	rs2'			0	1	And	c.and	rd', rs2'	and rd', rd', rs2'	rd' &= rs2'		
																1	0	0	1	1	1	– any –	– any –		– any –			0	1	**RESERVED on RV32 (c.subw / c.addw on RV64)**						
																0	1	0	i…	rd/rs1					imm			0	1	**Load Immediate**				**rd = imm**	**CI - Type** "Immediate"	**'C'**
– 5 –																										5	4:0			imm				CI sign-extended immediate		
																0	1	0	5	0	0	0	0	0	imm[4:0]			0	1	*HINT — c.li with rd = x0 (standard use) : PC += 2, no other effect*						
																0	1	0	5	rd ≠ 0					imm[4:0]			0	1	Load Immediate	c.li	rd, imm	addi rd, x0, imm	rd = imm		
																funct3			offset			rs1'			offset			0	1	**Branch**				**if (rs1' (funct3) x0) PC += offset**	**CB - Type** "Branch / Arithmetic"	**'C'**
– 8 –																							8	7:6	5	4:3	2:1	0			imm				CB sign-extended immediate		
																1	1	0	8	imm[…		rs1'			imm[…	imm[…	5	0	1	Branch if EQual to Zero	c.beqz	rs1', imm	beq rs1', x0, offset	if (rs1' == 0) PC += offset		
																1	1	1	8	imm[…		rs1'			imm[…	imm[…	5	0	1	Branch if Not Equal to Zero	c.bnez	rs1', imm	bne rs1', x0, offset	if (rs1' != 0) PC += offset		
																0	0	1	jump target									0	1	**Jump and Link**				**ra = PC + 2 ; PC += offset**	**CJ - Type** "Jump"	**'C'**
– 11 –																				11	10	9:8	7	6	5	4	3:1	0		imm				CJ sign-extended immediate		
																0	0	1	11	4	imm[…	10	6	7	imm[3:1]	5	0	1	Jump and Link	c.jal	imm	jal x1, offset	ra = PC + 2 ; PC += offset		
																1	0	1	jump target									0	1	**Jump**				**PC += offset**	**CJ - Type** "Jump"	**'C'**
– 11 –																				11	10	9:8	7	6	5	4	3:1	0		imm				CJ sign-extended immediate		
																1	0	1	11	4	imm[…	10	6	7	imm[3:1]	5	0	1	Jump	c.j	imm	jal x0, offset	PC += offset		
																0	1	1	i…	rd/rs1					imm			0	1	**Load Upper Immediate / Add to SP**				**rd = nzimm | sp += nzimm**	**CI - Type** "Immediate"	**'C'**
																0	1	1	0	– any –					0	0	0	0	0	1	**RESERVED (nzimm = 0)**						
																0	1	1	17	0	0	0	0	0	nzimm[16:12]			0	1	*HINT — c.lui with rd = x0 (standard use) : PC += 2, no other effect*						
– 17 –															17	16:12				– 0 –										nzimm				CI sign-extended immediate		
																0	1	1	9	0	0	0	1	0	4	6	nzim…	5	0	1	Add Immediate (×16) to SP	c.addi16sp	sp, imm	addi x2, x2, nzimm	sp += nzimm		
– 9 –																						9	8:7	6	5	4	– 0 –			nzimm				CI sign-extended immediate		
																0	1	1	17	rd ≠ 0, 2					nzimm[16:12]			0	1	Load Upper Immediate	c.lui	rd, imm	lui rd, nzimm	rd = nzimm (sign-extended)		
– 17 –															17	16:12				– 0 –										nzimm				CI sign-extended immediate		
Quadrant 3 — Non-Compressed Instructions (inst[1:0] = 11)																																					
imm[11:0]											rs1		funct3			rd									0	0	0	0	0	1	1	**Load**				**rd = M[rs1 + imm] [funct3]**	**I - Type** "Immediate"	**'I'**
– 11 –																11:0									imm								I sign-extended immediate		
imm[11:0]											rs1		0	0	0	rd									0	0	0	0	0	1	1	Load Byte	lb	rd, imm(rs1)		rd ← sext(M[rs1 + imm][7:0])		
imm[11:0]											rs1		1	0	0	rd									0	0	0	0	0	1	1	Load Byte Unsigned	lbu	rd, imm(rs1)		rd ← zext(M[rs1 + imm][7:0])		
imm[11:0]											rs1		0	1	0	rd									0	0	0	0	0	1	1	Load Word	lw	rd, imm(rs1)		rd ← M[rs1 + imm][31:0]		
imm[11:0]											rs1		0	0	1	rd									0	0	0	0	0	1	1	Load Half-Word	lh	rd, imm(rs1)		rd ← sext(M[rs1 + imm][15:0])		
imm[11:0]											rs1		1	0	1	rd									0	0	0	0	0	1	1	Load Half-Word Unsigned	lhu	rd, imm(rs1)		rd ← zext(M[rs1 + imm][15:0])		
rs3			funct2		rs2						rs1		funct3			rd									1	0	0	0	0	1	1	**Fused Multiply-Add**				**rd = rs1 × rs2 + rs3**	**R4 - Type** "Register (4 operands)"	**'F / D'**
rs3			0	0	rs2						rs1		rm			rd									1	0	0	0	0	1	1	Fused Multiply-Add	fmadd.s	rd, rs1, rs2, rs3, rm		rd = rs1 × rs2 + rs3		
rs3			0	1	rs2						rs1		rm			rd									1	0	0	0	0	1	1	Fused Multiply-Add	fmadd.d	rd, rs1, rs2, rs3, rm		rd = rs1 × rs2 + rs3		
imm[11:5]											rs2		rs1		funct3					imm[4:0]					0	1	0	0	0	1	1	**Store**				**rs2 [funct3] → M[rs1 + imm]**	**S - Type** "Store"	**'I'**
– 11 –																				11:5					4:0					imm				S sign-extended immediate		
imm[11:5]											rs2		rs1		0	0	0	imm[4:0]							0	1	0	0	0	1	1	Store Byte	sb	rs2, imm(rs1)		rs2[7:0] → M[rs1 + imm]		
imm[11:5]											rs2		rs1		0	1	0	imm[4:0]							0	1	0	0	0	1	1	Store Word	sw	rs2, imm(rs1)		rs2[31:0] → M[rs1 + imm]		
imm[11:5]											rs2		rs1		0	0	1	imm[4:0]							0	1	0	0	0	1	1	Store Half-Word	sh	rs2, imm(rs1)		rs2[15:0] → M[rs1 + imm]		
imm[12|10:5]											rs2		rs1		funct3			imm[4:1|11]							1	1	0	0	0	1	1	**Branch**				**if (rs1 (funct3) rs2) PC += imm**	**B - Type** "Branch"	**'I'**
– 12 –																			12	11	10:5						4:1		0			imm				B sign-extended immediate		
12	imm[10:5]										rs2		rs1		0	0	0	imm[4:1]	11						1	1	0	0	0	1	1	Branch if EQual	beq	rs1, rs2, imm		if (rs1 == rs2) PC += imm		
12	imm[10:5]										rs2		rs1		1	0	0	imm[4:1]	11						1	1	0	0	0	1	1	Branch if Less Than	blt	rs1, rs2, imm		if (rs1 < rs2) PC += imm (signed)		
12	imm[10:5]										rs2		rs1		1	1	0	imm[4:1]	11						1	1	0	0	0	1	1	Branch if Less Than Unsigned	bltu	rs1, rs2, imm		if (rs1 < rs2) PC += imm (unsigned)		
12	imm[10:5]										rs2		rs1		0	0	1	imm[4:1]	11						1	1	0	0	0	1	1	Branch if Not Equal	bne	rs1, rs2, imm		if (rs1 != rs2) PC += imm		
12	imm[10:5]										rs2		rs1		1	0	1	imm[4:1]	11						1	1	0	0	0	1	1	Branch if Greater or Equal	bge	rs1, rs2, imm		if (rs1 >= rs2) PC += imm (signed)		
12	imm[10:5]										rs2		rs1		1	1	1	imm[4:1]	11						1	1	0	0	0	1	1	Branch if Greater or Equal …	bgeu	rs1, rs2, imm		if (rs1 >= rs2) PC += imm (unsigned)		
imm[11:0]											rs1		funct3			rd									0	0	1	0	0	1	1	**Operations with Immediate**				**rd = rs1 (funct3) imm**	**I - Type** "Immediate"	**'I'**
imm[11:0]											rs1		0	0	0	rd									0	0	1	0	0	1	1	Add Immediate	addi	rd, rs1, imm		rd = rs1 + imm		
– 11 –																11:0									imm								I sign-extended immediate		
imm[11:0]											rs1		1	0	0	rd									0	0	1	0	0	1	1	Xor Immediate	xori	rd, rs1, imm		rd = rs1 ^ imm		
– 11 –																11:0									imm								I sign-extended immediate		
imm[11:0]											rs1		0	1	0	rd									0	0	1	0	0	1	1	Set Less Than Immediate	slti	rd, rs1, imm		rd = (rs1 < imm) ? 1 : 0 (signed)		
– 11 –																11:0									imm								I sign-extended immediate		

Bitwise Breakdown — inst[31:0]																												Meaning						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Instruction	Mnemonic	Operands	Expansion	Pseudocode	Type	Ext
imm[11:0]												rs1					**1**	**1**	**0**	rd					0	0	1	0	0	1	1	Or Immediate	**ori**	rd, rs1, imm		rd = rs1 | imm		
– *11* –												11:0																				imm				I sign-extended immediate		
0	**0**	**0**	**0**	**0**	**0**	**0**	shamt[4:0]					rs1					**0**	**0**	**1**	rd					0	0	1	0	0	1	1	Shift Left Logical Immediate	**slli**	rd, rs1, imm		rd = rs1 << shamt		
– 0 –																	4:0														shamt				I zero-extended immediate		
0	**0**	**0**	**0**	**0**	**0**	**0**	shamt[4:0]					rs1					**1**	**0**	**1**	rd					0	0	1	0	0	1	1	Shift Right Logical Immediate	**srli**	rd, rs1, imm		rd = rs1 >> shamt (zero extends)		
– 0 –																	4:0														shamt				I zero-extended immediate		
0	**1**	**0**	**0**	**0**	**0**	**0**	shamt[4:0]					rs1					**1**	**0**	**1**	rd					0	0	1	0	0	1	1	Shift Right Arithmetic Imm…	**srai**	rd, rs1, imm		rd = rs1 >> shamt (msb extends)		
– 0 –																	4:0														shamt				I zero-extended immediate		
imm[11:0]												rs1					**0**	**1**	**1**	rd					0	0	1	0	0	1	1	Set Less Than Immediate U…	**sltiu**	rd, rs1, imm		rd = (rs1 < imm) ? 1 : 0 (unsigned)		
– *11* –												11:0																				imm				I sign-extended immediate		
imm[11:0]												rs1					**1**	**1**	**1**	rd					0	0	1	0	0	1	1	And Immediate	**andi**	rd, rs1, imm		rd = rs1 & imm		
– *11* –												11:0																				imm				I sign-extended immediate		
funct7							rs2					rs1					**funct3**			rd					1	0	1	0	0	1	1	**Single-Precision Operations**				**rd = rs1 (funct7) rs2**	**R - Type** "Register"	**'F'**
0	**0**	**1**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**0**	**0**	rd					1	0	1	0	0	1	1	Sign Inject	**fsgnj.s**	rd, rs1, rs2		rd = { rs2[31], rs1[30:0] }		
1	**0**	**1**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**0**	**0**	rd					1	0	1	0	0	1	1	Less or Equal	**fle.s**	rd, rs1, rs2		x[rd] = (rs1 <= rs2)		
1	**1**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs1					**0**	**0**	**0**	rd					1	0	1	0	0	1	1	Move Float bits to Integer	**fmv.x.w**	rd, rs1		x[rd] = f[rs1][31:0]		
1	**1**	**1**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs1					**0**	**0**	**0**	rd					1	0	1	0	0	1	1	Move Integer bits to Float	**fmv.w.x**	rd, rs1		f[rd] = x[rs1][31:0]		
0	**0**	**1**	**0**	**1**	**0**	**0**	rs2					rs1					**0**	**0**	**0**	rd					1	0	1	0	0	1	1	Minimum	**fmin.s**	rd, rs1, rs2		rd = min(rs1, rs2)		
0	**0**	**1**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**1**	**0**	rd					1	0	1	0	0	1	1	Sign Inject Xor	**fsgnjx.s**	rd, rs1, rs2		rd = { rs1[31] ^ rs2[31], rs1[30:0] }		
1	**0**	**1**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**1**	**0**	rd					1	0	1	0	0	1	1	Equal	**feq.s**	rd, rs1, rs2		x[rd] = (rs1 == rs2)		
0	**0**	**1**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**0**	**1**	rd					1	0	1	0	0	1	1	Sign Inject Negated	**fsgnjn.s**	rd, rs1, rs2		rd = { ~rs2[31], rs1[30:0] }		
1	**0**	**1**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**0**	**1**	rd					1	0	1	0	0	1	1	Less Than	**flt.s**	rd, rs1, rs2		x[rd] = (rs1 < rs2)		
1	**1**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs1					**0**	**0**	**1**	rd					1	0	1	0	0	1	1	Classify	**fclass.s**	rd, rs1		x[rd] = class mask of f[rs1]		
0	**0**	**1**	**0**	**1**	**0**	**0**	rs2					rs1					**0**	**0**	**1**	rd					1	0	1	0	0	1	1	Maximum	**fmax.s**	rd, rs1, rs2		rd = max(rs1, rs2)		
0	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					rm			rd					1	0	1	0	0	1	1	Add	**fadd.s**	rd, rs1, rs2, rm		rd = rs1 + rs2		
1	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs1					rm			rd					1	0	1	0	0	1	1	Convert Float to Word	**fcvt.w.s**	rd, rs1, rm		x[rd] = (int32) f[rs1]		
1	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**1**	rs1					rm			rd					1	0	1	0	0	1	1	Convert Float to Unsigned …	**fcvt.wu.s**	rd, rs1, rm		x[rd] = (uint32) f[rs1]		
0	**0**	**0**	**1**	**0**	**0**	**0**	rs2					rs1					rm			rd					1	0	1	0	0	1	1	Multiply	**fmul.s**	rd, rs1, rs2, rm		rd = rs1 × rs2		
1	**1**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs1					rm			rd					1	0	1	0	0	1	1	Convert Word to Float	**fcvt.s.w**	rd, rs1, rm		f[rd] = (float) (int32) x[rs1]		
1	**1**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**1**	rs1					rm			rd					1	0	1	0	0	1	1	Convert Unsigned Word to …	**fcvt.s.wu**	rd, rs1, rm		f[rd] = (float) (uint32) x[rs1]		
0	**0**	**0**	**0**	**1**	**0**	**0**	rs2					rs1					rm			rd					1	0	1	0	0	1	1	Subtract	**fsub.s**	rd, rs1, rs2, rm		rd = rs1 − rs2		
0	**0**	**0**	**1**	**1**	**0**	**0**	rs2					rs1					rm			rd					1	0	1	0	0	1	1	Divide	**fdiv.s**	rd, rs1, rs2, rm		rd = rs1 ÷ rs2		
0	**1**	**0**	**1**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs1					rm			rd					1	0	1	0	0	1	1	Square Root	**fsqrt.s**	rd, rs1, rm		rd = √rs1		
funct7							rs2					rs1					**funct3**			rd					1	0	1	0	0	1	1	**Double-Precision Operations**				**rd = rs1 (funct7) rs2**		
0	**0**	**1**	**0**	**0**	**0**	**1**	rs2					rs1					**0**	**0**	**0**	rd					1	0	1	0	0	1	1	Sign Inject	**fsgnj.d**	rd, rs1, rs2		rd = { rs2[63], rs1[62:0] }		
1	**0**	**1**	**0**	**0**	**0**	**1**	rs2					rs1					**0**	**0**	**0**	rd					1	0	1	0	0	1	1	Less or Equal	**fle.d**	rd, rs1, rs2		x[rd] = (rs1 <= rs2)		
0	**0**	**1**	**0**	**1**	**0**	**1**	rs2					rs1					**0**	**0**	**0**	rd					1	0	1	0	0	1	1	Minimum	**fmin.d**	rd, rs1, rs2		rd = min(rs1, rs2)		
0	**0**	**1**	**0**	**0**	**0**	**1**	rs2					rs1					**0**	**1**	**0**	rd					1	0	1	0	0	1	1	Sign Inject Xor	**fsgnjx.d**	rd, rs1, rs2		rd = { rs1[63] ^ rs2[63], rs1[62:0] }		
1	**0**	**1**	**0**	**0**	**0**	**1**	rs2					rs1					**0**	**1**	**0**	rd					1	0	1	0	0	1	1	Equal	**feq.d**	rd, rs1, rs2		x[rd] = (rs1 == rs2)		
0	**0**	**1**	**0**	**0**	**0**	**1**	rs2					rs1					**0**	**0**	**1**	rd					1	0	1	0	0	1	1	Sign Inject Negated	**fsgnjn.d**	rd, rs1, rs2		rd = { ~rs2[63], rs1[62:0] }		
1	**0**	**1**	**0**	**0**	**0**	**1**	rs2					rs1					**0**	**0**	**1**	rd					1	0	1	0	0	1	1	Less Than	**flt.d**	rd, rs1, rs2		x[rd] = (rs1 < rs2)		
1	**1**	**1**	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	rs1					**0**	**0**	**1**	rd					1	0	1	0	0	1	1	Classify	**fclass.d**	rd, rs1		x[rd] = class mask of f[rs1]		
0	**0**	**1**	**0**	**1**	**0**	**1**	rs2					rs1					**0**	**0**	**1**	rd					1	0	1	0	0	1	1	Maximum	**fmax.d**	rd, rs1, rs2		rd = max(rs1, rs2)		
0	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**1**	rs1					rm			rd					1	0	1	0	0	1	1	Convert Double to Float	**fcvt.s.d**	rd, rs1, rm		f[rd] = (float) f[rs1]		
0	**0**	**0**	**0**	**0**	**0**	**1**	rs2					rs1					rm			rd					1	0	1	0	0	1	1	Add	**fadd.d**	rd, rs1, rs2, rm		rd = rs1 + rs2		
0	**1**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	rs1					rm			rd					1	0	1	0	0	1	1	Convert Float to Double	**fcvt.d.s**	rd, rs1		f[rd] = (double) f[rs1]		
1	**1**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	rs1					rm			rd					1	0	1	0	0	1	1	Convert Double to Word	**fcvt.w.d**	rd, rs1, rm		x[rd] = (int32) f[rs1]		
1	**1**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**1**	rs1					rm			rd					1	0	1	0	0	1	1	Convert Double to Unsigne…	**fcvt.wu.d**	rd, rs1, rm		x[rd] = (uint32) f[rs1]		
0	**0**	**0**	**1**	**0**	**0**	**1**	rs2					rs1					rm			rd					1	0	1	0	0	1	1	Multiply	**fmul.d**	rd, rs1, rs2, rm		rd = rs1 × rs2		
1	**1**	**0**	**1**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	rs1					rm			rd					1	0	1	0	0	1	1	Convert Word to Double	**fcvt.d.w**	rd, rs1		f[rd] = (double) (int32) x[rs1]		
1	**1**	**0**	**1**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**1**	rs1					rm			rd					1	0	1	0	0	1	1	Convert Unsigned Word to …	**fcvt.d.wu**	rd, rs1		f[rd] = (double) (uint32) x[rs1]		
0	**0**	**0**	**0**	**1**	**0**	**1**	rs2					rs1					rm			rd					1	0	1	0	0	1	1	Subtract	**fsub.d**	rd, rs1, rs2, rm		rd = rs1 − rs2		
0	**0**	**0**	**1**	**1**	**0**	**1**	rs2					rs1					rm			rd					1	0	1	0	0	1	1	Divide	**fdiv.d**	rd, rs1, rs2, rm		rd = rs1 ÷ rs2		
0	**1**	**0**	**1**	**1**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	rs1					rm			rd					1	0	1	0	0	1	1	Square Root	**fsqrt.d**	rd, rs1, rm		rd = √rs1		
funct7							rs2					rs1					**funct3**			rd					0	1	1	0	0	1	1	**Operations with Registers**				**rd = rs1 (funct) rs2**	**R - Type** "Register"	**'I'**
0	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**0**	**0**	rd					0	1	1	0	0	1	1	Add	**add**	rd, rs1, rs2		rd = rs1 + rs2		
0	**1**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**0**	**0**	rd					0	1	1	0	0	1	1	Subtract	**sub**	rd, rs1, rs2		rd = rs1 - rs2		
0	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**1**	**0**	**0**	rd					0	1	1	0	0	1	1	Xor	**xor**	rd, rs1, rs2		rd = rs1 ^ rs2		
0	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**1**	**0**	rd					0	1	1	0	0	1	1	Set Less Than	**slt**	rd, rs1, rs2		rd = (rs1 < rs2) ? 1 : 0 (signed)		
0	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**1**	**1**	**0**	rd					0	1	1	0	0	1	1	Or	**or**	rd, rs1, rs2		rd = rs1 | rs2		
0	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**0**	**1**	rd					0	1	1	0	0	1	1	Shift Left Logical	**sll**	rd, rs1, rs2		rd = rs1 << rs2[4:0]		
0	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**1**	**0**	**1**	rd					0	1	1	0	0	1	1	Shift Right Logical	**srl**	rd, rs1, rs2		rd = rs1 >> rs2[4:0] (zero extends)		
0	**1**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**1**	**0**	**1**	rd					0	1	1	0	0	1	1	Shift Right Arithmetic	**sra**	rd, rs1, rs2		rd = rs1 >> rs2[4:0] (msb extends)		
0	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**1**	**1**	rd					0	1	1	0	0	1	1	Set Less Than Unsigned	**sltu**	rd, rs1, rs2		rd = (rs1 < rs2) ? 1 : 0 (unsigned)		
0	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**1**	**1**	**1**	rd					0	1	1	0	0	1	1	And	**and**	rd, rs1, rs2		rd = rs1 & rs2		
0	**0**	**0**	**0**	**0**	**0**	**1**	rs2					rs1					**funct3**			rd					0	1	1	0	0	1	1	**Multiplication and Division**				**rd = rs1 (funct3) rs2**	**R - Type** "Register"	**'M'**
0	**0**	**0**	**0**	**0**	**0**	**1**	rs2					rs1					**0**	**0**	**0**	rd					0	1	1	0	0	1	1	Multiply	**mul**	rd, rs1, rs2		rd = (rs1 × rs2)[31:0]		
0	**0**	**0**	**0**	**0**	**0**	**1**	rs2					rs1					**1**	**0**	**0**	rd					0	1	1	0	0	1	1	Divide	**div**	rd, rs1, rs2		rd = rs1 ÷ rs2 (signed)		
0	**0**	**0**	**0**	**0**	**0**	**1**	rs2					rs1					**0**	**1**	**0**	rd					0	1	1	0	0	1	1	Multiply High Signed × Uns…	**mulhsu**	rd, rs1, rs2		rd = (s rs1 × u rs2)[63:32]		
0	**0**	**0**	**0**	**0**	**0**	**1**	rs2					rs1					**1**	**1**	**0**	rd					0	1	1	0	0	1	1	Remainder	**rem**	rd, rs1, rs2		rd = rs1 % rs2 (signed)		

Bitwise Breakdown — inst[31:0]																															Meaning						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	Instruction	Mnemonic	Operands	Expansion	Pseudocode	Type	Ext
0	**0**	**0**	**0**	**0**	**0**	**1**	rs2					rs1					**0**	**0**	**1**	rd					0	1	1	0	0	1	1	Multiply High Signed × Sig…	**mulh**	rd, rs1, rs2		rd = (s rs1 × s rs2)[63:32]		
0	**0**	**0**	**0**	**0**	**0**	**1**	rs2					rs1					**1**	**0**	**1**	rd					0	1	1	0	0	1	1	Divide Unsigned	**divu**	rd, rs1, rs2		rd = rs1 ÷ rs2 (unsigned)		
0	**0**	**0**	**0**	**0**	**0**	**1**	rs2					rs1					**0**	**1**	**1**	rd					0	1	1	0	0	1	1	Multiply High Unsigned × U…	**mulhu**	rd, rs1, rs2		rd = (u rs1 × u rs2)[63:32]		
0	**0**	**0**	**0**	**0**	**0**	**1**	rs2					rs1					**1**	**1**	**1**	rd					0	1	1	0	0	1	1	Remainder Unsigned	**remu**	rd, rs1, rs2		rd = rs1 % rs2 (unsigned)		
imm[11:0]												**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	1	1	1	0	0	1	1	**Environment**				**control to debug / OS**	**I - Type** "Immediate"	**'I'**
0	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	1	1	1	0	0	1	1	Environment Call	**ecall**	–		transfer control to execution environ…		
0	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	1	1	1	0	0	1	1	Environment Break	**ebreak**	–		transfer control back to debug enviro…		
imm[11:0]												rs1					**funct3**			rd					1	1	1	0	0	1	1	**Control and Status Registers**				**rd = CSR ; CSR = f(CSR, rs1 | zim…**	**I - Type** "Immediate"	**'Zicsr'**
– 0 –																											4:0					zimm				I zero-extended immediate		
csr[11:0]												rs1					**0**	**1**	**0**	rd					1	1	1	0	0	1	1	CSR Read and Set	**csrrs**	rd, csr, rs1		rd = CSR ; CSR |= rs1		
csr[11:0]												zimm[4:0]					**1**	**1**	**0**	rd					1	1	1	0	0	1	1	CSR Read and Set Immediate	**csrrsi**	rd, csr, imm		rd = CSR ; CSR |= zimm		
csr[11:0]												rs1					**0**	**0**	**1**	rd					1	1	1	0	0	1	1	CSR Read and Write	**csrrw**	rd, csr, rs1		rd = CSR ; CSR = rs1		
csr[11:0]												zimm[4:0]					**1**	**0**	**1**	rd					1	1	1	0	0	1	1	CSR Read and Write Imme…	**csrrwi**	rd, csr, imm		rd = CSR ; CSR = zimm		
csr[11:0]												rs1					**0**	**1**	**1**	rd					1	1	1	0	0	1	1	CSR Read and Clear	**csrrc**	rd, csr, rs1		rd = CSR ; CSR &= ~rs1		
csr[11:0]												zimm[4:0]					**1**	**1**	**1**	rd					1	1	1	0	0	1	1	CSR Read and Clear Immed…	**csrrci**	rd, csr, imm		rd = CSR ; CSR &= ~zimm		
– not used in RV32GC –																									0	0	0	1	0	1	1	**custom-0**				designated for custom extensions	—	—
rs3					**funct2**		rs2					rs1					**funct3**			rd					1	0	0	1	0	1	1	**Fused Negated Multiply-Subtract**				**rd = −(rs1 × rs2) + rs3**	**R4 - Type** "Register (4 operands)"	**'F / D'**
rs3			**0**	**0**			rs2					rs1					rm			rd					1	0	0	1	0	1	1	Fused Negated Multiply-Su…	**fnmsub.s**	rd, rs1, rs2, rs3, rm		rd = −(rs1 × rs2) + rs3		
rs3			**0**	**1**			rs2					rs1					rm			rd					1	0	0	1	0	1	1	Fused Negated Multiply-Su…	**fnmsub.d**	rd, rs1, rs2, rs3, rm		rd = −(rs1 × rs2) + rs3		
– not used in RV32GC –																									0	1	0	1	0	1	1	**custom-1**				designated for custom extensions	—	—
– not used in RV32GC –																									1	1	0	1	0	1	1	**reserved**				reserved for future standard extensions	—	—
– not used in RV32GC –																									0	0	1	1	0	1	1	**OP-IMM-32**				RV64 only (addiw, sliw, …)	—	—
– not used in RV32GC –																									1	0	1	1	0	1	1	**custom-2**				designated for custom extensions	—	—
– not used in RV32GC –																									0	1	1	1	0	1	1	**OP-32**				RV64 only (addw, subw, …)	—	—
– not used in RV32GC –																									1	1	1	1	0	1	1	**custom-3**				designated for custom extensions	—	—
imm[11:0]												rs1					**funct3**			rd					0	0	0	0	1	1	1	**Floating-Point Load**				**rd = M[rs1 + imm] [funct3]**	**I - Type** "Immediate"	**'F / D'**
– *11* –																									11:0							imm				I sign-extended immediate		
imm[11:0]												rs1					**0**	**1**	**0**	rd					0	0	0	0	1	1	1	Load Float	**flw**	rd, imm(rs1)		rd ← M[rs1 + imm][31:0]		
imm[11:0]												rs1					**0**	**1**	**1**	rd					0	0	0	0	1	1	1	Load Double	**fld**	rd, imm(rs1)		rd ← M[rs1 + imm][63:0]		
rs3			**funct2**				rs2					rs1					**funct3**			rd					1	0	0	0	1	1	1	**Fused Multiply-Subtract**				**rd = rs1 × rs2 − rs3**	**R4 - Type** "Register (4 operands)"	**'F / D'**
rs3			**0**	**0**			rs2					rs1					rm			rd					1	0	0	0	1	1	1	Fused Multiply-Subtract	**fmsub.s**	rd, rs1, rs2, rs3, rm		rd = rs1 × rs2 − rs3		
rs3			**0**	**1**			rs2					rs1					rm			rd					1	0	0	0	1	1	1	Fused Multiply-Subtract	**fmsub.d**	rd, rs1, rs2, rs3, rm		rd = rs1 × rs2 − rs3		
imm[11:5]							rs2					rs1					**funct3**			imm[4:0]					0	1	0	0	1	1	1	**Floating-Point Store**				**rs2 [funct3] → M[rs1 + imm]**	**S - Type** "Store"	**'F / D'**
– *11* –																									11:5							imm				S sign-extended immediate		
imm[11:5]							rs2					rs1					**0**	**1**	**0**	imm[4:0]					0	1	0	0	1	1	1	Store Float	**fsw**	rs2, imm(rs1)		rs2[31:0] → M[rs1 + imm]		
imm[11:5]							rs2					rs1					**0**	**1**	**1**	imm[4:0]					0	1	0	0	1	1	1	Store Double	**fsd**	rs2, imm(rs1)		rs2[63:0] → M[rs1 + imm]		
imm[11:0]												rs1					**0**	**0**	**0**	rd					1	1	0	0	1	1	1	**Jump and Link Register**				**rd = PC + 4 ; PC = rs1 + imm**	**I - Type** "Immediate"	**'I'**
– *11* –																									11:0							imm				I sign-extended immediate		
imm[11:0]												rs1					**0**	**0**	**0**	rd					1	1	0	0	1	1	1	Jump and Link Register	**jalr**	rd, imm(rs1)		rd = PC + 4 ; PC = (rs1 + imm) & ~1		
imm[31:12]																				rd					0	0	1	0	1	1	1	**Add Upper Immediate to PC**				**rd = PC + imm**	**U - Type** "Upper Immediate"	**'I'**
31:12																				– 0 –												imm				U sign-extended immediate		
imm[31:12]																				rd					0	0	1	0	1	1	1	Add Upper Immediate to PC	**auipc**	rd, imm		rd = PC + (imm << 12)		
– not used in RV32GC –																									1	0	1	0	1	1	1	**OP-V**				'V' vector extension	—	—
imm[31:12]																				rd					0	1	1	0	1	1	1	**Load Upper Immediate**				**rd = imm**	**U - Type** "Upper Immediate"	**'I'**
31:12																				– 0 –												imm				U sign-extended immediate		
imm[31:12]																				rd					0	1	1	0	1	1	1	Load Upper Immediate	**lui**	rd, imm		rd = imm << 12		
– not used in RV32GC –																									1	1	1	0	1	1	1	**OP-VE**				vector crypto extensions (Zvk*)	—	—
imm[11:0]												rs1					**0**	**0**	**0**	rd					0	0	0	1	1	1	1	**Memory Ordering**				**order memory accesses**	**I - Type** "Immediate"	**'I'**
0	**0**	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	0	0	0	1	1	1	1	*PAUSE (Zihintpause) — fence w, 0: briefly pause instruction retirement : PC += 4, no other effect*						
1	**0**	**0**	**0**	**0**	**0**	**1**	**1**	**0**	**0**	**1**	**1**	– any –					**0**	**0**	**0**	– any –					0	0	0	1	1	1	1	Fence (Total Store Order)	**fence.tso**	–		order rw → rw, except w → r		
0	**0**	**0**	**0**	pred			succ					– any –					**0**	**0**	**0**	– any –					0	0	0	1	1	1	1	Fence	**fence**	pred, succ		order pred accesses before succ acce…		
– any –			– any –				– any –					– any –					**0**	**0**	**0**	– any –					0	0	0	1	1	1	1	**RESERVED fm / pred / succ — executes as a normal fence**						
imm[11:0]												rs1					**0**	**0**	**1**	rd					0	0	0	1	1	1	1	**Instruction Fence**				**sync instruction and data streams**	**I - Type** "Immediate"	**'Zifenc…**
– any –												– any –					**0**	**0**	**1**	– any –					0	0	0	1	1	1	1	Instruction Fence	**fence.i**	–		sync instruction and data streams		
rs3					**funct2**		rs2					rs1					**funct3**			rd					1	0	0	1	1	1	1	**Fused Negated Multiply-Add**				**rd = −(rs1 × rs2) − rs3**	**R4 - Type** "Register (4 operands)"	**'F / D'**
rs3			**0**	**0**			rs2					rs1					rm			rd					1	0	0	1	1	1	1	Fused Negated Multiply-Add	**fnmadd.s**	rd, rs1, rs2, rs3, rm		rd = −(rs1 × rs2) − rs3		
rs3			**0**	**1**			rs2					rs1					rm			rd					1	0	0	1	1	1	1	Fused Negated Multiply-Add	**fnmadd.d**	rd, rs1, rs2, rs3, rm		rd = −(rs1 × rs2) − rs3		
funct7							rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	**Atomic Operations**				**t = M[rs1] ; M[rs1] = t (funct5) rs…**	**R - Type** "Register"	**'A'**
0	**0**	**0**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	Atomic Add	**amoadd.w**	rd, rs2, (rs1)		t = M[rs1] ; M[rs1] = t + rs2 ; rd = t		
1	**0**	**0**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	Atomic Min	**amomin.w**	rd, rs2, (rs1)		t = M[rs1] ; M[rs1] = min(t, rs2) (sig…		
0	**1**	**0**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	Atomic Or	**amoor.w**	rd, rs2, (rs1)		t = M[rs1] ; M[rs1] = t | rs2 ; rd = t		
1	**1**	**0**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	Atomic Min Unsigned	**amominu.w**	rd, rs2, (rs1)		t = M[rs1] ; M[rs1] = min(t, rs2) (uns…		
0	**0**	**1**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	Atomic Xor	**amoxor.w**	rd, rs2, (rs1)		t = M[rs1] ; M[rs1] = t ^ rs2 ; rd = t		
1	**0**	**1**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	Atomic Max	**amomax.w**	rd, rs2, (rs1)		t = M[rs1] ; M[rs1] = max(t, rs2) (sig…		
0	**1**	**1**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	Atomic And	**amoand.w**	rd, rs2, (rs1)		t = M[rs1] ; M[rs1] = t & rs2 ; rd = t		
1	**1**	**1**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	Atomic Max Unsigned	**amomaxu.w**	rd, rs2, (rs1)		t = M[rs1] ; M[rs1] = max(t, rs2) (un…		
0	**0**	**0**	**1**	**0**	aq	rl	**0**	**0**	**0**	**0**	**0**	rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	Load Reserved	**lr.w**	rd, (rs1)		rd ← M[rs1] ; reserve M[rs1]		
0	**0**	**0**	**0**	**1**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	Atomic Swap	**amoswap.w**	rd, rs2, (rs1)		t = M[rs1] ; M[rs1] = rs2 ; rd = t		
0	**0**	**0**	**1**	**1**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1	Store Conditional	**sc.w**	rd, rs2, (rs1)		if reserved { M[rs1] = rs2 ; rd = 0 } el…		

Bitwise Breakdown — inst[31:0]																												Meaning						
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	**Instruction**	**Mnemonic**	**Operands**	**Expansion**	**Pseudocode**	**Type**	**Ext**
imm[20|10:1|11|19:12]																				rd					1	1	0	1	1	1	**Jump and Link**				**rd = PC + 4 ; PC += imm**	**J - Type** "Jump"	**'I'**
– 20 –											20	19:12								11	10:1									0	imm				J sign-extended immediate		
20	imm[10:1]										11	imm[19:12]								rd					1	1	0	1	1	1	Jump and Link	`jal`	`rd, imm`		rd = PC + 4 ; PC += imm		
– not used in RV32GC –																									0	0	1	1	1	1	1	**reserved (>32b)**				reserved for instructions longer than ...	—	—
– not used in RV32GC –																									1	0	1	1	1	1	1	**reserved (>32b)**				reserved for instructions longer than ...	—	—
– not used in RV32GC –																									0	1	1	1	1	1	1	**reserved (>32b)**				reserved for instructions longer than ...	—	—
– not used in RV32GC –																									1	1	1	1	1	1	1	**reserved (>32b)**				reserved for instructions longer than ...	—	—

RV32GC

Sliderule Encoder

Instruction Set "Sliderule" Encoder by Simone Shawn Cazzaniga (magiwanders@github | www.magiwanders.com | shawn@magiwanders.com)

Meaning				Assembly		Bitwise Encoding — inst[31:0]																												
Ext	**Type**	**Description**	**Explanation**	**Mnemonic**	**Operands**	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I	**R - Type** "Register"	Add	rd = rs1 + rs2	**add**	rd, rs1, rs2	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**0**	**0**	rd					0	1	1	0	0	1	1
I	**I - Type** "Immediate"	Add Immediate	rd = rs1 + imm	**addi**	rd, rs1, imm	imm[11:0]												rs1					**0**	**0**	**0**	rd					0	0	1	0	0	1	1
			imm			*– 11 –*																				11:0										
A	**R - Type** "Register"	Atomic Add	t = M[rs1] ; M[rs1] = t + rs2 ; rd = t	**amoadd.w**	rd, rs2, (rs1)	**0**	**0**	**0**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1
A	**R - Type** "Register"	Atomic And	t = M[rs1] ; M[rs1] = t & rs2 ; rd = t	**amoand.w**	rd, rs2, (rs1)	**0**	**1**	**1**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1
A	**R - Type** "Register"	Atomic Max	t = M[rs1] ; M[rs1] = max(t, rs2) (sign…	**amomax.w**	rd, rs2, (rs1)	**1**	**0**	**1**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1
A	**R - Type** "Register"	Atomic Max Unsigned	t = M[rs1] ; M[rs1] = max(t, rs2) (unsi…	**amomaxu.w**	rd, rs2, (rs1)	**1**	**1**	**1**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1
A	**R - Type** "Register"	Atomic Min	t = M[rs1] ; M[rs1] = min(t, rs2) (signe…	**amomin.w**	rd, rs2, (rs1)	**1**	**0**	**0**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1
A	**R - Type** "Register"	Atomic Min Unsigned	t = M[rs1] ; M[rs1] = min(t, rs2) (unsig…	**amominu.w**	rd, rs2, (rs1)	**1**	**1**	**0**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1
A	**R - Type** "Register"	Atomic Or	t = M[rs1] ; M[rs1] = t | rs2 ; rd = t	**amoor.w**	rd, rs2, (rs1)	**0**	**1**	**0**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1
A	**R - Type** "Register"	Atomic Swap	t = M[rs1] ; M[rs1] = rs2 ; rd = t	**amoswap.w**	rd, rs2, (rs1)	**0**	**0**	**0**	**0**	**1**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1
A	**R - Type** "Register"	Atomic Xor	t = M[rs1] ; M[rs1] = t ^ rs2 ; rd = t	**amoxor.w**	rd, rs2, (rs1)	**0**	**0**	**1**	**0**	**0**	aq	rl	rs2					rs1					**0**	**1**	**0**	rd					0	1	0	1	1	1	1
I	**R - Type** "Register"	And	rd = rs1 & rs2	**and**	rd, rs1, rs2	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**1**	**1**	**1**	rd					0	1	1	0	0	1	1
I	**I - Type** "Immediate"	And Immediate	rd = rs1 & imm	**andi**	rd, rs1, imm	imm[11:0]												rs1					**1**	**1**	**1**	rd					0	0	1	0	0	1	1
			imm			*– 11 –*																				11:0										
I	**U - Type** "Upper Immediate"	Add Upper Immediate to PC	rd = PC + (imm << 12)	**auipc**	rd, imm	imm[31:12]																				rd					0	0	1	0	1	1	1
			imm			31:12																				– 0 –										
I	**B - Type** "Branch"	Branch if EQual	if (rs1 == rs2) PC += imm	**beq**	rs1, rs2, imm	12	imm[10:5]						rs2					rs1					**0**	**0**	**0**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
PSEUDO	**B - Type** "Branch"	Branch if = Zero	beq rs, x0, imm	**beqz**	rs, imm	12	imm[10:5]						**0**	**0**	**0**	**0**	**0**	rs					**0**	**0**	**0**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
I	**B - Type** "Branch"	Branch if Greater or Equal	if (rs1 >= rs2) PC += imm (signed)	**bge**	rs1, rs2, imm	12	imm[10:5]						rs2					rs1					**1**	**0**	**1**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
I	**B - Type** "Branch"	Branch if Greater or Equal Unsigned	if (rs1 >= rs2) PC += imm (unsigned)	**bgeu**	rs1, rs2, imm	12	imm[10:5]						rs2					rs1					**1**	**1**	**1**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
PSEUDO	**B - Type** "Branch"	Branch if ≥ Zero	bge rs, x0, imm	**bgez**	rs, imm	12	imm[10:5]						**0**	**0**	**0**	**0**	**0**	rs					**1**	**0**	**1**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
PSEUDO	**B - Type** "Branch"	Branch if Greater	blt rt, rs, imm	**bgt**	rs, rt, imm	12	imm[10:5]						rs					rt					**1**	**0**	**0**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
PSEUDO	**B - Type** "Branch"	Branch if Greater Unsigned	bltu rt, rs, imm	**bgtu**	rs, rt, imm	12	imm[10:5]						rs					rt					**1**	**1**	**0**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
PSEUDO	**B - Type** "Branch"	Branch if > Zero	blt x0, rs, imm	**bgtz**	rs, imm	12	imm[10:5]						rs					**0**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
PSEUDO	**B - Type** "Branch"	Branch if Less or Equal	bge rt, rs, imm	**ble**	rs, rt, imm	12	imm[10:5]						rs					rt					**1**	**0**	**1**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
PSEUDO	**B - Type** "Branch"	Branch if Less or Equal Unsigned	bgeu rt, rs, imm	**bleu**	rs, rt, imm	12	imm[10:5]						rs					rt					**1**	**1**	**1**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
PSEUDO	**B - Type** "Branch"	Branch if ≤ Zero	bge x0, rs, imm	**blez**	rs, imm	12	imm[10:5]						rs					**0**	**0**	**0**	**0**	**0**	**1**	**0**	**1**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
I	**B - Type** "Branch"	Branch if Less Than	if (rs1 < rs2) PC += imm (signed)	**blt**	rs1, rs2, imm	12	imm[10:5]						rs2					rs1					**1**	**0**	**0**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
I	**B - Type** "Branch"	Branch if Less Than Unsigned	if (rs1 < rs2) PC += imm (unsigned)	**bltu**	rs1, rs2, imm	12	imm[10:5]						rs2					rs1					**1**	**1**	**0**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
PSEUDO	**B - Type** "Branch"	Branch if < Zero	blt rs, x0, imm	**bltz**	rs, imm	12	imm[10:5]						**0**	**0**	**0**	**0**	**0**	rs					**1**	**0**	**0**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
I	**B - Type** "Branch"	Branch if Not Equal	if (rs1 != rs2) PC += imm	**bne**	rs1, rs2, imm	12	imm[10:5]						rs2					rs1					**0**	**0**	**1**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
PSEUDO	**B - Type** "Branch"	Branch if ≠ Zero	bne rs, x0, imm	**bnez**	rs, imm	12	imm[10:5]						**0**	**0**	**0**	**0**	**0**	rs					**0**	**0**	**1**	imm[4:1]				11	1	1	0	0	0	1	1
			imm			*– 12 –*																			12	11	10:5					4:1				0
C	**CR - Type** "Register"	Add	add rd, rd, rs2 rd += rs2	**c.add**	rd, rs2																		**1**	**0**	**0**	**1**	rd/rs1 ≠ 0				rs2 ≠ 0				1	0
C	**CI - Type** "Immediate"	Add Immediate	addi rd, rd, nzimm rd += nzimm	**c.addi**	rd, imm																		**0**	**0**	**0**	5	rd/rs1 ≠ 0				nzimm[4:0]				0	1
			nzimm			*– 5 –*																									5	4:0				
C	**CI - Type** "Immediate"	Add Immediate (×16) to SP	addi x2, x2, nzimm sp += nzimm	**c.addi16sp**	sp, imm																		**0**	**1**	**1**	9	**0**	**0**	**0**	**1**	**0**	4	6	nzim…	5	0	1
			nzimm			*– 9 –*																								9	8:7	6	5	4	– 0 –				
C	**CIW - Type** "Wide Immediate"	Add Immediate (×4) to SP into rd'	addi rd', sp, nzuimm rd' = sp + nzuimm	**c.addi4spn**	rd', sp, imm																		**0**	**0**	**0**	nzui…				nzuimm[9:6]				2	3	rd'			0	0

Meaning				Assembly		Bitwise Encoding — inst[31:0]
Ext	Type	Description	Explanation	Mnemonic	Operands	31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
			nzuimm			– 0 – | 9:6 | 5:4 | 3 | 2 | – 0 –
C	**CA - Type** "Arithmetic"	And	and rd', rd', rs2' rd' &= rs2'	c.and	rd', rs2'	[redacted] | **1** | **0** | **0** | **0** | **1** | **1** | rd'/rs1' | **1** | **1** | rs2' | 0 | 1
C	**CB - Type** "Branch / Arithmetic"	And Immediate	andi rd', rd', imm rd' &= imm	c.andi	rd', imm	[redacted] | **1** | **0** | **0** | 5 | **1** | **0** | rd'/rs1' | imm[4:0] | 0 | 1
			imm			– 5 – | 5 | 4:0
C	**CB - Type** "Branch / Arithmetic"	Branch if EQual to Zero	beq rs1', x0, offset if (rs1' == 0) PC += offset	c.beqz	rs1', imm	[redacted] | **1** | **1** | **0** | 8 | imm[… | rs1' | imm[… | imm[… | 5 | 0 | 1
			imm			– 8 – | 8 | 7:6 | 5 | 4:3 | 2:1 | 0
C	**CB - Type** "Branch / Arithmetic"	Branch if Not Equal to Zero	bne rs1', x0, offset if (rs1' != 0) PC += offset	c.bnez	rs1', imm	[redacted] | **1** | **1** | **1** | 8 | imm[… | rs1' | imm[… | imm[… | 5 | 0 | 1
			imm			– 8 – | 8 | 7:6 | 5 | 4:3 | 2:1 | 0
C	**CR - Type** "Register"	Environment Break	ebreak control back to debug environment	c.ebreak	–	[redacted] | **1** | **0** | **0** | **1** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | 1 | 0
C + D	**CL - Type** "Load"	Load Double (FP)	fld rd', uimm(rs1') rd' ← M[rs1' + uimm][63:0]	c.fld	rd', imm(rs1')	[redacted] | **0** | **0** | **1** | uimm[5:3] | rs1' | uim… | rd' | 0 | 0
			uimm			– 0 – | 7:6 | 5:3 | – 0 –
C + D	**CI - Type** "Immediate"	Load Double from SP (FP)	fld rd, uimm(sp) rd ← M[sp + uimm][63:0]	c.fldsp	rd, imm(sp)	[redacted] | **0** | **0** | **1** | 5 | rd | uim… | uimm[8:6] | 1 | 0
			uimm			– 0 – | 8:6 | 5 | 4:3 | – 0 –
C + F	**CL - Type** "Load"	Load Float (FP)	flw rd', uimm(rs1') rd' ← M[rs1' + uimm][31:0]	c.flw	rd', imm(rs1')	[redacted] | **0** | **1** | **1** | uimm[5:3] | rs1' | 2 | 6 | rd' | 0 | 0
			uimm			– 0 – | 6 | 5:3 | 2 | – 0 –
C + F	**CI - Type** "Immediate"	Load Float from SP (FP)	flw rd, uimm(sp) rd ← M[sp + uimm][31:0]	c.flwsp	rd, imm(sp)	[redacted] | **0** | **1** | **1** | 5 | rd | uimm[4:2] | uim… | 1 | 0
			uimm			– 0 – | 7:6 | 5 | 4:2 | – 0 –
C + D	**CS - Type** "Store"	Store Double (FP)	fsd rs2', uimm(rs1') rs2' → M[rs1' + uimm][63:0]	c.fsd	rs2', imm(rs1')	[redacted] | **1** | **0** | **1** | uimm[5:3] | rs1' | uim… | rs2' | 0 | 0
			uimm			– 0 – | 7:6 | 5:3 | – 0 –
C + D	**CSS - Type** "Stack-relative Store"	Store Double to SP (FP)	fsd rs2, uimm(sp) rs2 → M[sp + uimm][63:0]	c.fsdsp	rs2, imm(sp)	[redacted] | **1** | **0** | **1** | uimm[5:3] | uimm[8:6] | rs2 | 1 | 0
			uimm			– 0 – | 8:6 | 5:3 | – 0 –
C + F	**CS - Type** "Store"	Store Float (FP)	fsw rs2', uimm(rs1') rs2' → M[rs1' + uimm][31:0]	c.fsw	rs2', imm(rs1')	[redacted] | **1** | **1** | **1** | uimm[5:3] | rs1' | 2 | 6 | rs2' | 0 | 0
			uimm			– 0 – | 6 | 5:3 | 2 | – 0 –
C + F	**CSS - Type** "Stack-relative Store"	Store Float to SP (FP)	fsw rs2, uimm(sp) rs2 → M[sp + uimm][31:0]	c.fswsp	rs2, imm(sp)	[redacted] | **1** | **1** | **1** | uimm[5:2] | uim… | rs2 | 1 | 0
			uimm			– 0 – | 7:6 | 5:2 | – 0 –
C	**CJ - Type** "Jump"	Jump	jal x0, offset PC += offset	c.j	imm	[redacted] | **1** | **0** | **1** | 11 | 4 | imm[… | 10 | 6 | 7 | imm[3:1] | 5 | 0 | 1
			imm			– 11 – | 11 | 10 | 9:8 | 7 | 6 | 5 | 4 | 3:1 | 0
C	**CJ - Type** "Jump"	Jump and Link	jal x1, offset ra = PC + 2 ; PC += offset	c.jal	imm	[redacted] | **0** | **0** | **1** | 11 | 4 | imm[… | 10 | 6 | 7 | imm[3:1] | 5 | 0 | 1
			imm			– 11 – | 11 | 10 | 9:8 | 7 | 6 | 5 | 4 | 3:1 | 0
C	**CR - Type** "Register"	Jump and Link Register	jalr x1, 0(rs1) ra = PC + 2 ; PC = rs1	c.jalr	rs1	[redacted] | **1** | **0** | **0** | **1** | rs1 ≠ 0 | **0** | **0** | **0** | **0** | **0** | 1 | 0
C	**CR - Type** "Register"	Jump Register	jalr x0, 0(rs1) PC = rs1	c.jr	rs1	[redacted] | **1** | **0** | **0** | **0** | rs1 ≠ 0 | **0** | **0** | **0** | **0** | **0** | 1 | 0
C	**CI - Type** "Immediate"	Load Immediate	addi rd, x0, imm rd = imm	c.li	rd, imm	[redacted] | **0** | **1** | **0** | 5 | rd ≠ 0 | imm[4:0] | 0 | 1
			imm			– 5 – | 5 | 4:0
C	**CI - Type** "Immediate"	Load Upper Immediate	lui rd, nzimm rd = nzimm (sign-extended)	c.lui	rd, imm	[redacted] | **0** | **1** | **1** | 17 | rd ≠ 0, 2 | nzimm[16:12] | 0 | 1
			nzimm			– 17 – | 17 | 16:12 | – 0 –
C	**CL - Type** "Load"	Load Word	lw rd', uimm(rs1') rd' ← M[rs1' + uimm][31:0]	c.lw	rd', imm(rs1')	[redacted] | **0** | **1** | **0** | uimm[5:3] | rs1' | 2 | 6 | rd' | 0 | 0
			uimm			– 0 – | 6 | 5:3 | 2 | – 0 –
C	**CI - Type** "Immediate"	Load Word from SP	lw rd, uimm(sp) rd ← M[sp + uimm][31:0]	c.lwsp	rd, imm(sp)	[redacted] | **0** | **1** | **0** | 5 | rd ≠ 0 | uimm[4:2] | uim… | 1 | 0
			uimm			– 0 – | 7:6 | 5 | 4:2 | – 0 –
C	**CR - Type** "Register"	Move Register	add rd, x0, rs2 rd = rs2	c.mv	rd, rs2	[redacted] | **1** | **0** | **0** | **0** | rd ≠ 0 | rs2 ≠ 0 | 1 | 0
C	**CI - Type** "Immediate"	No Operation	addi x0, x0, 0 literally do nothing	c.nop	–	[redacted] | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | **0** | 0 | 1
C	**CA - Type** "Arithmetic"	Or	or rd', rd', rs2' rd' |= rs2'	c.or	rd', rs2'	[redacted] | **1** | **0** | **0** | **0** | **1** | **1** | rd'/rs1' | **1** | **0** | rs2' | 0 | 1
C	**CI - Type** "Immediate"	Shift Left Logical Immediate	slli rd, rd, shamt rd <<= shamt	c.slli	rd, imm	[redacted] | **0** | **0** | **0** | **0** | rd/rs1 ≠ 0 | shamt[4:0] | 1 | 0
			shamt			– 0 – | 4:0
C	**CB - Type** "Branch / Arithmetic"	Shift Right Arithmetic Immediate	srai rd', rd', shamt rd' = rd' >> shamt (msb extends)	c.srai	rd', imm	[redacted] | **1** | **0** | **0** | **0** | **0** | **1** | rd'/rs1' | shamt[4:0] | 0 | 1
			shamt			– 0 – | 4:0
C	**CB - Type** "Branch / Arithmetic"	Shift Right Logical Immediate	srli rd', rd', shamt rd' = rd' >> shamt (zero extends)	c.srli	rd', imm	[redacted] | **1** | **0** | **0** | **0** | **0** | **0** | rd'/rs1' | shamt[4:0] | 0 | 1
			shamt			– 0 – | 4:0
C	**CA - Type** "Arithmetic"	Subtract	sub rd', rd', rs2' rd' -= rs2'	c.sub	rd', rs2'	[redacted] | **1** | **0** | **0** | **0** | **1** | **1** | rd'/rs1' | **0** | **0** | rs2' | 0 | 1

Meaning				Assembly		Bitwise Encoding — inst[31:0]
Ext	Type	Description	Explanation	Mnemonic	Operands	31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
C	**CS - Type** "Store"	Store Word	sw rs2', uimm(rs1') rs2' → M[rs1' + uimm][31:0]	c.sw	rs2', imm(rs1')	[redacted] 1 1 0 uimm[5:3] rs1' 2 6 rs2' 0 0
			uimm			– 0 – 6 5:3 2 – 0 –
C	**CSS - Type** "Stack-relative Store"	Store Word to SP	sw rs2, uimm(sp) rs2 → M[sp + uimm][31:0]	c.swsp	rs2, imm(sp)	[redacted] 1 1 0 uimm[5:2] uim… rs2 1 0
			uimm			– 0 – 7:6 5:2 – 0 –
C	**CA - Type** "Arithmetic"	Xor	xor rd', rd', rs2' rd' ^= rs2'	c.xor	rd', rs2'	[redacted] 1 0 0 0 1 1 rd'/rs1' 0 1 rs2' 0 1
PSEUDO	/	Call Far-Away Subroutine	auipc x1, imm[31:12]	call	imm	imm[31:12] 0 0 0 0 0 1 0 1 1 1 1 1
			jalr x1, imm[11:0](x1)			imm[11:0] 0 0 0 0 1 0 0 0 0 0 0 0 1 1 1 0 0 1 1 1
PSEUDO	**I - Type** "Immediate"	CSR Clear bits	csrrc x0, csr, rs	csrc	csr, rs	csr[11:0] rs 0 1 1 0 0 0 0 0 1 1 1 0 0 1 1
PSEUDO	**I - Type** "Immediate"	CSR Clear bits Immediate	csrrci x0, csr, imm	csrci	csr, imm	csr[11:0] zimm[4:0] 1 1 1 0 0 0 0 0 1 1 1 0 0 1 1
			zimm			– 0 – 4:0
PSEUDO	**I - Type** "Immediate"	CSR Read	csrrs rd, csr, x0	csrr	rd, csr	csr[11:0] 0 0 0 0 0 0 1 0 rd 1 1 1 0 0 1 1
Zicsr	**I - Type** "Immediate"	CSR Read and Clear	rd = CSR ; CSR &= ~rs1	csrrc	rd, csr, rs1	csr[11:0] rs1 0 1 1 rd 1 1 1 0 0 1 1
Zicsr	**I - Type** "Immediate"	CSR Read and Clear Immediate	rd = CSR ; CSR &= ~zimm	csrrci	rd, csr, imm	csr[11:0] zimm[4:0] 1 1 1 rd 1 1 1 0 0 1 1
			zimm			– 0 – 4:0
Zicsr	**I - Type** "Immediate"	CSR Read and Set	rd = CSR ; CSR |= rs1	csrrs	rd, csr, rs1	csr[11:0] rs1 0 1 0 rd 1 1 1 0 0 1 1
Zicsr	**I - Type** "Immediate"	CSR Read and Set Immediate	rd = CSR ; CSR |= zimm	csrrsi	rd, csr, imm	csr[11:0] zimm[4:0] 1 1 0 rd 1 1 1 0 0 1 1
			zimm			– 0 – 4:0
Zicsr	**I - Type** "Immediate"	CSR Read and Write	rd = CSR ; CSR = rs1	csrrw	rd, csr, rs1	csr[11:0] rs1 0 0 1 rd 1 1 1 0 0 1 1
Zicsr	**I - Type** "Immediate"	CSR Read and Write Immediate	rd = CSR ; CSR = zimm	csrrwi	rd, csr, imm	csr[11:0] zimm[4:0] 1 0 1 rd 1 1 1 0 0 1 1
			zimm			– 0 – 4:0
PSEUDO	**I - Type** "Immediate"	CSR Set bits	csrrs x0, csr, rs	csrs	csr, rs	csr[11:0] rs 0 1 0 0 0 0 0 0 1 1 1 0 0 1 1
PSEUDO	**I - Type** "Immediate"	CSR Set bits Immediate	csrrsi x0, csr, imm	csrsi	csr, imm	csr[11:0] zimm[4:0] 1 1 0 0 0 0 0 0 1 1 1 0 0 1 1
			zimm			– 0 – 4:0
PSEUDO	**I - Type** "Immediate"	CSR Write	csrrw x0, csr, rs	csrw	csr, rs	csr[11:0] rs 0 0 1 0 0 0 0 0 1 1 1 0 0 1 1
PSEUDO	**I - Type** "Immediate"	CSR Write Immediate	csrrwi x0, csr, imm	csrwi	csr, imm	csr[11:0] zimm[4:0] 1 0 1 0 0 0 0 0 1 1 1 0 0 1 1
			zimm			– 0 – 4:0
M	**R - Type** "Register"	Divide	rd = rs1 ÷ rs2 (signed)	div	rd, rs1, rs2	0 0 0 0 0 0 1 rs2 rs1 1 0 0 rd 0 1 1 0 0 1 1
M	**R - Type** "Register"	Divide Unsigned	rd = rs1 ÷ rs2 (unsigned)	divu	rd, rs1, rs2	0 0 0 0 0 0 1 rs2 rs1 1 0 1 rd 0 1 1 0 0 1 1
I	**I - Type** "Immediate"	Environment Break	transfer control back to debug environ…	ebreak	–	0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 1 1
I	**I - Type** "Immediate"	Environment Call	transfer control to execution environment	ecall	–	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 1 1
PSEUDO	**R - Type** "Register"	Double Absolute Value	fsgnjx.d rd, rs, rs	fabs.d	rd, rs	0 0 1 0 0 0 1 rs rs 0 1 0 rd 1 0 1 0 0 1 1
PSEUDO	**R - Type** "Register"	Float Absolute Value	fsgnjx.s rd, rs, rs	fabs.s	rd, rs	0 0 1 0 0 0 0 rs rs 0 1 0 rd 1 0 1 0 0 1 1
D	**R - Type** "Register"	Add	rd = rs1 + rs2	fadd.d	rd, rs1, rs2, rm	0 0 0 0 0 0 1 rs2 rs1 rm rd 1 0 1 0 0 1 1
F	**R - Type** "Register"	Add	rd = rs1 + rs2	fadd.s	rd, rs1, rs2, rm	0 0 0 0 0 0 0 rs2 rs1 rm rd 1 0 1 0 0 1 1
D	**R - Type** "Register"	Classify	x[rd] = class mask of f[rs1]	fclass.d	rd, rs1	1 1 1 0 0 0 1 0 0 0 0 0 rs1 0 0 1 rd 1 0 1 0 0 1 1
F	**R - Type** "Register"	Classify	x[rd] = class mask of f[rs1]	fclass.s	rd, rs1	1 1 1 0 0 0 0 0 0 0 0 0 rs1 0 0 1 rd 1 0 1 0 0 1 1
D	**R - Type** "Register"	Convert Float to Double	f[rd] = (double) f[rs1]	fcvt.d.s	rd, rs1	0 1 0 0 0 0 1 0 0 0 0 0 rs1 rm rd 1 0 1 0 0 1 1
D	**R - Type** "Register"	Convert Word to Double	f[rd] = (double) (int32) x[rs1]	fcvt.d.w	rd, rs1	1 1 0 1 0 0 1 0 0 0 0 0 rs1 rm rd 1 0 1 0 0 1 1
D	**R - Type** "Register"	Convert Unsigned Word to Double	f[rd] = (double) (uint32) x[rs1]	fcvt.d.wu	rd, rs1	1 1 0 1 0 0 1 0 0 0 0 1 rs1 rm rd 1 0 1 0 0 1 1
D	**R - Type** "Register"	Convert Double to Float	f[rd] = (float) f[rs1]	fcvt.s.d	rd, rs1, rm	0 1 0 0 0 0 0 0 0 0 0 1 rs1 rm rd 1 0 1 0 0 1 1
F	**R - Type** "Register"	Convert Word to Float	f[rd] = (float) (int32) x[rs1]	fcvt.s.w	rd, rs1, rm	1 1 0 1 0 0 0 0 0 0 0 0 rs1 rm rd 1 0 1 0 0 1 1
F	**R - Type** "Register"	Convert Unsigned Word to Float	f[rd] = (float) (uint32) x[rs1]	fcvt.s.wu	rd, rs1, rm	1 1 0 1 0 0 0 0 0 0 0 1 rs1 rm rd 1 0 1 0 0 1 1
D	**R - Type** "Register"	Convert Double to Word	x[rd] = (int32) f[rs1]	fcvt.w.d	rd, rs1, rm	1 1 0 0 0 0 1 0 0 0 0 0 rs1 rm rd 1 0 1 0 0 1 1
F	**R - Type** "Register"	Convert Float to Word	x[rd] = (int32) f[rs1]	fcvt.w.s	rd, rs1, rm	1 1 0 0 0 0 0 0 0 0 0 0 rs1 rm rd 1 0 1 0 0 1 1
D	**R - Type** "Register"	Convert Double to Unsigned Word	x[rd] = (uint32) f[rs1]	fcvt.wu.d	rd, rs1, rm	1 1 0 0 0 0 1 0 0 0 0 1 rs1 rm rd 1 0 1 0 0 1 1
F	**R - Type** "Register"	Convert Float to Unsigned Word	x[rd] = (uint32) f[rs1]	fcvt.wu.s	rd, rs1, rm	1 1 0 0 0 0 0 0 0 0 0 1 rs1 rm rd 1 0 1 0 0 1 1
D	**R - Type** "Register"	Divide	rd = rs1 ÷ rs2	fdiv.d	rd, rs1, rs2, rm	0 0 0 1 1 0 1 rs2 rs1 rm rd 1 0 1 0 0 1 1

Meaning				Assembly		Bitwise Encoding — inst[31:0]																														
Ext	Type	Description	Explanation	Mnemonic	Operands	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
F	R - Type "Register"	Divide	rd = rs1 ÷ rs2	fdiv.s	rd, rs1, rs2, rm	0	0	0	1	1	0	0	rs2					rs1					rm			rd					1	0	1	0	0	1	1
I	I - Type "Immediate"	Fence	order pred accesses before succ accesses	fence	pred, succ	0	0	0	0	pred					succ			– any –					0	0	0	– any –				0	0	0	1	1	1	1
PSEUDO	I - Type "Immediate"	Fence (all accesses)	fence iorw, iorw	fence	–	0	0	0	0	1	1	1	1	1	1	1	1	– any –					0	0	0	– any –				0	0	0	1	1	1	1
Zifencei	I - Type "Immediate"	Instruction Fence	sync instruction and data streams	fence.i	–	– any –												– any –					0	0	1	– any –				0	0	0	1	1	1	1
I	I - Type "Immediate"	Fence (Total Store Order)	order rw → rw, except w → r	fence.tso	–	1	0	0	0	0	0	1	1	0	0	1	1	– any –					0	0	0	– any –				0	0	0	1	1	1	1
D	R - Type "Register"	Equal	x[rd] = (rs1 == rs2)	feq.d	rd, rs1, rs2	1	0	1	0	0	0	1	rs2					rs1					0	1	0	rd				1	0	1	0	0	1	1
F	R - Type "Register"	Equal	x[rd] = (rs1 == rs2)	feq.s	rd, rs1, rs2	1	0	1	0	0	0	0	rs2					rs1					0	1	0	rd				1	0	1	0	0	1	1
D	I - Type "Immediate"	Load Double	rd ← M[rs1 + imm][63:0]	fld	rd, imm(rs1)	imm[11:0]												rs1					0	1	1	rd				0	0	0	0	1	1	1
			imm			– 11 –																				11:0										
PSEUDO	/	Load Global Double	auipc rt, imm[31:12]	fld	rd, imm, rt	imm[31:12]																				rt				0	0	1	0	1	1	1
			fld rd, imm[11:0](rt)			imm[11:0]												rt					0	1	1	rd				0	0	0	0	1	1	1
D	R - Type "Register"	Less or Equal	x[rd] = (rs1 <= rs2)	fle.d	rd, rs1, rs2	1	0	1	0	0	0	1	rs2					rs1					0	0	0	rd				1	0	1	0	0	1	1
F	R - Type "Register"	Less or Equal	x[rd] = (rs1 <= rs2)	fle.s	rd, rs1, rs2	1	0	1	0	0	0	0	rs2					rs1					0	0	0	rd				1	0	1	0	0	1	1
D	R - Type "Register"	Less Than	x[rd] = (rs1 < rs2)	flt.d	rd, rs1, rs2	1	0	1	0	0	0	1	rs2					rs1					0	0	1	rd				1	0	1	0	0	1	1
F	R - Type "Register"	Less Than	x[rd] = (rs1 < rs2)	flt.s	rd, rs1, rs2	1	0	1	0	0	0	0	rs2					rs1					0	0	1	rd				1	0	1	0	0	1	1
F	I - Type "Immediate"	Load Float	rd ← M[rs1 + imm][31:0]	flw	rd, imm(rs1)	imm[11:0]												rs1					0	1	0	rd				0	0	0	0	1	1	1
			imm			– 11 –																				11:0										
PSEUDO	/	Load Global Float	auipc rt, imm[31:12]	flw	rd, imm, rt	imm[31:12]																				rt				0	0	1	0	1	1	1
			flw rd, imm[11:0](rt)			imm[11:0]												rt					0	1	0	rd				0	0	0	0	1	1	1
D	R4 - Type "Register (4 operands)"	Fused Multiply-Add	rd = rs1 × rs2 + rs3	fmadd.d	rd, rs1, rs2, rs3, rm	rs3					0	1	rs2					rs1					rm			rd				1	0	0	0	0	1	1
F	R4 - Type "Register (4 operands)"	Fused Multiply-Add	rd = rs1 × rs2 + rs3	fmadd.s	rd, rs1, rs2, rs3, rm	rs3					0	0	rs2					rs1					rm			rd				1	0	0	0	0	1	1
D	R - Type "Register"	Maximum	rd = max(rs1, rs2)	fmax.d	rd, rs1, rs2	0	0	1	0	1	0	1	rs2					rs1					0	0	1	rd				1	0	1	0	0	1	1
F	R - Type "Register"	Maximum	rd = max(rs1, rs2)	fmax.s	rd, rs1, rs2	0	0	1	0	1	0	0	rs2					rs1					0	0	1	rd				1	0	1	0	0	1	1
D	R - Type "Register"	Minimum	rd = min(rs1, rs2)	fmin.d	rd, rs1, rs2	0	0	1	0	1	0	1	rs2					rs1					0	0	0	rd				1	0	1	0	0	1	1
F	R - Type "Register"	Minimum	rd = min(rs1, rs2)	fmin.s	rd, rs1, rs2	0	0	1	0	1	0	0	rs2					rs1					0	0	0	rd				1	0	1	0	0	1	1
D	R4 - Type "Register (4 operands)"	Fused Multiply-Subtract	rd = rs1 × rs2 − rs3	fmsub.d	rd, rs1, rs2, rs3, rm	rs3					0	1	rs2					rs1					rm			rd				1	0	0	0	1	1	1
F	R4 - Type "Register (4 operands)"	Fused Multiply-Subtract	rd = rs1 × rs2 − rs3	fmsub.s	rd, rs1, rs2, rs3, rm	rs3					0	0	rs2					rs1					rm			rd				1	0	0	0	1	1	1
D	R - Type "Register"	Multiply	rd = rs1 × rs2	fmul.d	rd, rs1, rs2, rm	0	0	0	1	0	0	1	rs2					rs1					rm			rd				1	0	1	0	0	1	1
F	R - Type "Register"	Multiply	rd = rs1 × rs2	fmul.s	rd, rs1, rs2, rm	0	0	0	1	0	0	0	rs2					rs1					rm			rd				1	0	1	0	0	1	1
PSEUDO	R - Type "Register"	Copy Double Register	fsgnj.d rd, rs, rs	fmv.d	rd, rs	0	0	1	0	0	0	1	rs					rs					0	0	0	rd				1	0	1	0	0	1	1
PSEUDO	R - Type "Register"	Copy Float Register	fsgnj.s rd, rs, rs	fmv.s	rd, rs	0	0	1	0	0	0	0	rs					rs					0	0	0	rd				1	0	1	0	0	1	1
F	R - Type "Register"	Move Integer bits to Float	f[rd] = x[rs1][31:0]	fmv.w.x	rd, rs1	1	1	1	1	0	0	0	0	0	0	0	0	rs1					0	0	0	rd				1	0	1	0	0	1	1
F	R - Type "Register"	Move Float bits to Integer	x[rd] = f[rs1][31:0]	fmv.x.w	rd, rs1	1	1	1	0	0	0	0	0	0	0	0	0	rs1					0	0	0	rd				1	0	1	0	0	1	1
PSEUDO	R - Type "Register"	Double Negate	fsgnjn.d rd, rs, rs	fneg.d	rd, rs	0	0	1	0	0	0	1	rs					rs					0	0	1	rd				1	0	1	0	0	1	1
PSEUDO	R - Type "Register"	Float Negate	fsgnjn.s rd, rs, rs	fneg.s	rd, rs	0	0	1	0	0	0	0	rs					rs					0	0	1	rd				1	0	1	0	0	1	1
D	R4 - Type "Register (4 operands)"	Fused Negated Multiply-Add	rd = −(rs1 × rs2) − rs3	fnmadd.d	rd, rs1, rs2, rs3, rm	rs3					0	1	rs2					rs1					rm			rd				1	0	0	1	1	1	1
F	R4 - Type "Register (4 operands)"	Fused Negated Multiply-Add	rd = −(rs1 × rs2) − rs3	fnmadd.s	rd, rs1, rs2, rs3, rm	rs3					0	0	rs2					rs1					rm			rd				1	0	0	1	1	1	1
D	R4 - Type "Register (4 operands)"	Fused Negated Multiply-Subtract	rd = −(rs1 × rs2) + rs3	fnmsub.d	rd, rs1, rs2, rs3, rm	rs3					0	1	rs2					rs1					rm			rd				1	0	0	1	0	1	1
F	R4 - Type "Register (4 operands)"	Fused Negated Multiply-Subtract	rd = −(rs1 × rs2) + rs3	fnmsub.s	rd, rs1, rs2, rs3, rm	rs3					0	0	rs2					rs1					rm			rd				1	0	0	1	0	1	1
PSEUDO	I - Type "Immediate"	Read FP Control/Status Register	csrrs rd, fcsr, x0	frcsr	rd	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	1	0	rd				1	1	1	0	0	1	1
PSEUDO	I - Type "Immediate"	Read FP Exception Flags	csrrs rd, fflags, x0	frflags	rd	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	rd				1	1	1	0	0	1	1

Meaning				Assembly		Bitwise Encoding — inst[31:0]
Ext	Type	Description	Explanation	Mnemonic	Operands	31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
PSEUDO	I - Type "Immediate"	Read FP Rounding Mode	csrrs rd, frm, x0	frrm	rd	0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 rd 1 1 1 0 0 1 1
PSEUDO	I - Type "Immediate"	Swap FP Control/Status Register	csrrw rd, fcsr, rs	fscsr	rd, rs	0 0 0 0 0 0 0 0 0 0 1 1 rs 0 0 1 rd 1 1 1 0 0 1 1
PSEUDO	I - Type "Immediate"	Write FP Control/Status Register	csrrw x0, fcsr, rs	fscsr	rs	0 0 0 0 0 0 0 0 0 0 1 1 rs 0 0 1 0 0 0 0 0 1 1 1 0 0 1 1
D	S - Type "Store"	Store Double	rs2[63:0] → M[rs1 + imm] imm	fsd	rs2, imm(rs1)	imm[11:5] rs2 rs1 0 1 1 imm[4:0] 0 1 0 0 1 1 1 – 11 – 11:5 4:0
PSEUDO	/	Store Global Double	auipc rt, imm[31:12] fsd rs2, imm[11:0](rt)	fsd	rs2, imm, rt	imm[31:12] rt 0 0 1 0 1 1 1 imm[11:5] rs2 rt 0 1 1 imm[4:0] 0 1 0 0 1 1 1
PSEUDO	I - Type "Immediate"	Swap FP Exception Flags	csrrw rd, fflags, rs	fsflags	rd, rs	0 0 0 0 0 0 0 0 0 0 0 1 rs 0 0 1 rd 1 1 1 0 0 1 1
PSEUDO	I - Type "Immediate"	Write FP Exception Flags	csrrw x0, fflags, rs	fsflags	rs	0 0 0 0 0 0 0 0 0 0 0 1 rs 0 0 1 0 0 0 0 0 1 1 1 0 0 1 1
PSEUDO	I - Type "Immediate"	Swap FP Exception Flags, Immediate	csrrwi rd, fflags, imm zimm	fsflagsi	rd, imm	0 0 0 0 0 0 0 0 0 0 0 1 zimm[4:0] 1 0 1 rd 1 1 1 0 0 1 1 – 0 – 4:0
PSEUDO	I - Type "Immediate"	Write FP Exception Flags, Immediate	csrrwi x0, fflags, imm zimm	fsflagsi	imm	0 0 0 0 0 0 0 0 0 0 0 1 zimm[4:0] 1 0 1 0 0 0 0 0 1 1 1 0 0 1 1 – 0 – 4:0
D	R - Type "Register"	Sign Inject	rd = { rs2[63], rs1[62:0] }	fsgnj.d	rd, rs1, rs2	0 0 1 0 0 0 1 rs2 rs1 0 0 0 rd 1 0 1 0 0 1 1
F	R - Type "Register"	Sign Inject	rd = { rs2[31], rs1[30:0] }	fsgnj.s	rd, rs1, rs2	0 0 1 0 0 0 0 rs2 rs1 0 0 0 rd 1 0 1 0 0 1 1
D	R - Type "Register"	Sign Inject Negated	rd = { ~rs2[63], rs1[62:0] }	fsgnjn.d	rd, rs1, rs2	0 0 1 0 0 0 1 rs2 rs1 0 0 1 rd 1 0 1 0 0 1 1
F	R - Type "Register"	Sign Inject Negated	rd = { ~rs2[31], rs1[30:0] }	fsgnjn.s	rd, rs1, rs2	0 0 1 0 0 0 0 rs2 rs1 0 0 1 rd 1 0 1 0 0 1 1
D	R - Type "Register"	Sign Inject Xor	rd = { rs1[63] ^ rs2[63], rs1[62:0] }	fsgnjx.d	rd, rs1, rs2	0 0 1 0 0 0 1 rs2 rs1 0 1 0 rd 1 0 1 0 0 1 1
F	R - Type "Register"	Sign Inject Xor	rd = { rs1[31] ^ rs2[31], rs1[30:0] }	fsgnjx.s	rd, rs1, rs2	0 0 1 0 0 0 0 rs2 rs1 0 1 0 rd 1 0 1 0 0 1 1
D	R - Type "Register"	Square Root	rd = √rs1	fsqrt.d	rd, rs1, rm	0 1 0 1 1 0 1 0 0 0 0 0 rs1 rm rd 1 0 1 0 0 1 1
F	R - Type "Register"	Square Root	rd = √rs1	fsqrt.s	rd, rs1, rm	0 1 0 1 1 0 0 0 0 0 0 0 rs1 rm rd 1 0 1 0 0 1 1
PSEUDO	I - Type "Immediate"	Swap FP Rounding Mode	csrrw rd, frm, rs	fsrm	rd, rs	0 0 0 0 0 0 0 0 0 0 1 0 rs 0 0 1 rd 1 1 1 0 0 1 1
PSEUDO	I - Type "Immediate"	Write FP Rounding Mode	csrrw x0, frm, rs	fsrm	rs	0 0 0 0 0 0 0 0 0 0 1 0 rs 0 0 1 0 0 0 0 0 1 1 1 0 0 1 1
PSEUDO	I - Type "Immediate"	Swap FP Rounding Mode, Immediate	csrrwi rd, frm, imm zimm	fsrmi	rd, imm	0 0 0 0 0 0 0 0 0 0 1 0 zimm[4:0] 1 0 1 rd 1 1 1 0 0 1 1 – 0 – 4:0
PSEUDO	I - Type "Immediate"	Write FP Rounding Mode, Immediate	csrrwi x0, frm, imm zimm	fsrmi	imm	0 0 0 0 0 0 0 0 0 0 1 0 zimm[4:0] 1 0 1 0 0 0 0 0 1 1 1 0 0 1 1 – 0 – 4:0
D	R - Type "Register"	Subtract	rd = rs1 − rs2	fsub.d	rd, rs1, rs2, rm	0 0 0 0 1 0 1 rs2 rs1 rm rd 1 0 1 0 0 1 1
F	R - Type "Register"	Subtract	rd = rs1 − rs2	fsub.s	rd, rs1, rs2, rm	0 0 0 0 1 0 0 rs2 rs1 rm rd 1 0 1 0 0 1 1
F	S - Type "Store"	Store Float	rs2[31:0] → M[rs1 + imm] imm	fsw	rs2, imm(rs1)	imm[11:5] rs2 rs1 0 1 0 imm[4:0] 0 1 0 0 1 1 1 – 11 – 11:5 4:0
PSEUDO	/	Store Global Float	auipc rt, imm[31:12] fsw rs2, imm[11:0](rt)	fsw	rs2, imm, rt	imm[31:12] rt 0 0 1 0 1 1 1 imm[11:5] rs2 rt 0 1 0 imm[4:0] 0 1 0 0 1 1 1
PSEUDO	J - Type "Jump"	Jump	jal x0, imm imm	j	imm	20 imm[10:1] 11 imm[19:12] 0 0 0 0 0 1 1 0 1 1 1 – 20 – 20 19:12 11 10:1 0
I	J - Type "Jump"	Jump and Link	rd = PC + 4 ; PC += imm imm	jal	rd, imm	20 imm[10:1] 11 imm[19:12] rd 1 1 0 1 1 1 1 – 20 – 20 19:12 11 10:1 0
PSEUDO	J - Type "Jump"	Jump and Link (to ra)	jal x1, imm imm	jal	imm	20 imm[10:1] 11 imm[19:12] 0 0 0 0 1 1 1 0 1 1 1 1 – 20 – 20 19:12 11 10:1 0
I	I - Type "Immediate"	Jump and Link Register	rd = PC + 4 ; PC = (rs1 + imm) & ~1 imm	jalr	rd, imm(rs1)	imm[11:0] rs1 0 0 0 rd 1 1 0 0 1 1 1 – 11 – 11:0
PSEUDO	I - Type "Immediate"	Jump and Link Register (to ra)	jalr x1, 0(rs)	jalr	rs	0 0 0 0 0 0 0 0 0 0 0 0 rs 0 0 0 0 0 0 0 1 1 1 0 0 1 1 1
PSEUDO	I - Type "Immediate"	Jump Register	jalr x0, 0(rs)	jr	rs	0 0 0 0 0 0 0 0 0 0 0 0 rs 0 0 0 0 0 0 0 0 1 1 0 0 1 1 1
PSEUDO	/	Load Address (PC-relative)	auipc rd, imm[31:12] addi rd, rd, imm[11:0]	la	rd, imm	imm[31:12] rd 0 0 1 0 1 1 1 imm[11:0] rd 0 0 0 rd 0 0 1 0 0 1 1
I	I - Type "Immediate"	Load Byte	rd ← sext(M[rs1 + imm][7:0]) imm	lb	rd, imm(rs1)	imm[11:0] rs1 0 0 0 rd 0 0 0 0 0 1 1 – 11 – 11:0
PSEUDO	/	Load Global Byte	auipc rd, imm[31:12] lb rd, imm[11:0](rd)	lb	rd, imm	imm[31:12] rd 0 0 1 0 1 1 1 imm[11:0] rd 0 0 0 rd 0 0 0 0 0 1 1
I	I - Type "Immediate"	Load Byte Unsigned	rd ← zext(M[rs1 + imm][7:0]) imm	lbu	rd, imm(rs1)	imm[11:0] rs1 1 0 0 rd 0 0 0 0 0 1 1 – 11 – 11:0
I	I - Type "Immediate"	Load Half-Word	rd ← sext(M[rs1 + imm][15:0]) imm	lh	rd, imm(rs1)	imm[11:0] rs1 0 0 1 rd 0 0 0 0 0 1 1 – 11 – 11:0
PSEUDO	/	Load Global Half-Word	auipc rd, imm[31:12] lh rd, imm[11:0](rd)	lh	rd, imm	imm[31:12] rd 0 0 1 0 1 1 1 imm[11:0] rd 0 0 1 rd 0 0 0 0 0 1 1
I	I - Type "Immediate"	Load Half-Word Unsigned	rd ← zext(M[rs1 + imm][15:0])	lhu	rd, imm(rs1)	imm[11:0] rs1 1 0 1 rd 0 0 0 0 0 1 1

Meaning				Assembly		Bitwise Encoding — inst[31:0]																													
Ext	Type	Description	Explanation	Mnemonic	Operands	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
			imm			– *11* –																							11:0						
PSEUDO	/	Load Immediate (just addi rd, x0, im…	lui rd, imm[31:12]	li	rd, imm	imm[31:12]																							rd						
			addi rd, rd, imm[11:0]			imm[11:0]												rd					0	0	0			rd						
PSEUDO	/	Load Local Address (always PC-relati…	auipc rd, imm[31:12]	lla	rd, imm	imm[31:12]																							rd						
			addi rd, rd, imm[11:0]			imm[11:0]												rd					0	0	0			rd						
A	R - Type "Register"	Load Reserved	rd ← M[rs1] ; reserve M[rs1]	lr.w	rd, (rs1)	0	0	0	1	0	aq	rl	0	0	0	0	0	rs1					0	1	0			rd						
I	U - Type "Upper Immediate"	Load Upper Immediate	rd = imm << 12	lui	rd, imm	imm[31:12]																							rd						
			imm			31:12																							– 0 –						
I	I - Type "Immediate"	Load Word	rd ← M[rs1 + imm][31:0]	lw	rd, imm(rs1)	imm[11:0]												rs1					0	1	0			rd						
			imm			– *11* –																							11:0						
PSEUDO	/	Load Global Word	auipc rd, imm[31:12]	lw	rd, imm	imm[31:12]																							rd						
			lw rd, imm[11:0](rd)			imm[11:0]												rd					0	1	0			rd						
M	R - Type "Register"	Multiply	rd = (rs1 × rs2)[31:0]	mul	rd, rs1, rs2	0	0	0	0	0	0	1	rs2					rs1					0	0	0			rd						
M	R - Type "Register"	Multiply High Signed × Signed	rd = (s rs1 × s rs2)[63:32]	mulh	rd, rs1, rs2	0	0	0	0	0	0	1	rs2					rs1					0	0	1			rd						
M	R - Type "Register"	Multiply High Signed × Unsigned	rd = (s rs1 × u rs2)[63:32]	mulhsu	rd, rs1, rs2	0	0	0	0	0	0	1	rs2					rs1					0	1	0			rd						
M	R - Type "Register"	Multiply High Unsigned × Unsigned	rd = (u rs1 × u rs2)[63:32]	mulhu	rd, rs1, rs2	0	0	0	0	0	0	1	rs2					rs1					0	1	1			rd						
PSEUDO	I - Type "Immediate"	Move Register	addi rd, rs, 0	mv	rd, rs	0	0	0	0	0	0	0	0	0	0	0	0	rs					0	0	0			rd						
PSEUDO	R - Type "Register"	2's Complement	sub rd, x0, rs	neg	rd, rs	0	1	0	0	0	0	0	rs					0	0	0	0	0	0	0	0			rd						
PSEUDO	I - Type "Immediate"	No Operation	addi x0, x0, 0	nop	–	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1
PSEUDO	I - Type "Immediate"	1's Complement	xori rd, rs, -1	not	rd, rs	1	1	1	1	1	1	1	1	1	1	1	1	rs					1	0	0			rd						
I	R - Type "Register"	Or	rd = rs1 | rs2	or	rd, rs1, rs2	0	0	0	0	0	0	0	rs2					rs1					1	1	0			rd						
I	I - Type "Immediate"	Or Immediate	rd = rs1 | imm	ori	rd, rs1, imm	imm[11:0]												rs1					1	1	0			rd						
			imm			– *11* –																							11:0						
PSEUDO	I - Type "Immediate"	Pause Hint (Zihintpause)	fence w, 0	pause	–	0	0	0	0	0	0	0	1	0	0	0	0	– any –					0	0	0	– any –					0	0	0	1	1	1	1
PSEUDO	I - Type "Immediate"	Read Cycle Counter	csrrs rd, cycle, x0	rdcycle	rd	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0			rd			1	1	1	0	0	1	1
PSEUDO	I - Type "Immediate"	Read Cycle Counter, upper 32 bits	csrrs rd, cycleh, x0	rdcycleh	rd	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0			rd			1	1	1	0	0	1	1
PSEUDO	I - Type "Immediate"	Read Instructions-Retired Counter	csrrs rd, instret, x0	rdinstret	rd	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0			rd			1	1	1	0	0	1	1
PSEUDO	I - Type "Immediate"	Read Instructions-Retired Counter, u…	csrrs rd, instreth, x0	rdinstreth	rd	1	1	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0			rd			1	1	1	0	0	1	1
PSEUDO	I - Type "Immediate"	Read Real-Time Clock	csrrs rd, time, x0	rdtime	rd	1	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0			rd			1	1	1	0	0	1	1
PSEUDO	I - Type "Immediate"	Read Real-Time Clock, upper 32 bits	csrrs rd, timeh, x0	rdtimeh	rd	1	1	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0			rd			1	1	1	0	0	1	1
M	R - Type "Register"	Remainder	rd = rs1 % rs2 (signed)	rem	rd, rs1, rs2	0	0	0	0	0	0	1	rs2					rs1					1	1	0			rd						
M	R - Type "Register"	Remainder Unsigned	rd = rs1 % rs2 (unsigned)	remu	rd, rs1, rs2	0	0	0	0	0	0	1	rs2					rs1					1	1	1			rd						
PSEUDO	I - Type "Immediate"	Return from Subroutine	jalr x0, 0(x1)	ret	–	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	1	0	0	1	1	1
I	S - Type "Store"	Store Byte	rs2[7:0] → M[rs1 + imm]	sb	rs2, imm(rs1)	imm[11:5]							rs2					rs1					0	0	0	imm[4:0]					0	1	0	0	0	1	1
			imm			– *11* –																				11:5					4:0					
PSEUDO	/	Store Global Byte	auipc rt, imm[31:12]	sb	rs2, imm, rt	imm[31:12]																							rt						
			sb rs2, imm[11:0](rt)			imm[11:5]							rs2					rt					0	0	0	imm[4:0]					0	1	0	0	0	1	1
A	R - Type "Register"	Store Conditional	if reserved { M[rs1] = rs2 ; rd = 0 } els…	sc.w	rd, rs2, (rs1)	0	0	0	1	1	aq	rl	rs2					rs1					0	1	0			rd						
PSEUDO	I - Type "Immediate"	Set if = Zero	sltiu rd, rs, 1	seqz	rd, rs	0	0	0	0	0	0	0	0	0	0	0	1	rs					0	1	1			rd						
PSEUDO	R - Type "Register"	Set if > Zero	slt rd, x0, rs	sgtz	rd, rs	0	0	0	0	0	0	0	rs					0	0	0	0	0	0	1	0			rd						
I	S - Type "Store"	Store Half-Word	rs2[15:0] → M[rs1 + imm]	sh	rs2, imm(rs1)	imm[11:5]							rs2					rs1					0	0	1	imm[4:0]					0	1	0	0	0	1	1
			imm			– *11* –																				11:5					4:0					
PSEUDO	/	Store Global Half-Word	auipc rt, imm[31:12]	sh	rs2, imm, rt	imm[31:12]																							rt						
			sh rs2, imm[11:0](rt)			imm[11:5]							rs2					rt					0	0	1	imm[4:0]					0	1	0	0	0	1	1
I	R - Type "Register"	Shift Left Logical	rd = rs1 << rs2[4:0]	sll	rd, rs1, rs2	0	0	0	0	0	0	0	rs2					rs1					0	0	1	rd					0	1	1	0	0	1	1
I	I - Type "Immediate"	Shift Left Logical Immediate	rd = rs1 << shamt	slli	rd, rs1, imm	0	0	0	0	0	0	0	shamt[4:0]					rs1					0	0	1	rd					0	0	1	0	0	1	1
			shamt			– 0 –																							4:0						
I	R - Type "Register"	Set Less Than	rd = (rs1 < rs2) ? 1 : 0 (signed)	slt	rd, rs1, rs2	0	0	0	0	0	0	0	rs2					rs1					0	1	0	rd					0	1	1	0	0	1	1
I	I - Type "Immediate"	Set Less Than Immediate	rd = (rs1 < imm) ? 1 : 0 (signed)	slti	rd, rs1, imm	imm[11:0]												rs1					0	1	0	rd					0	0	1	0	0	1	1
			imm			– *11* –																				11:0										
I	I - Type "Immediate"	Set Less Than Immediate Unsigned	rd = (rs1 < imm) ? 1 : 0 (unsigned)	sltiu	rd, rs1, imm	imm[11:0]												rs1					0	1	1	rd					0	0	1	0	0	1	1
			imm			– *11* –																				11:0										

Meaning				Assembly		Bitwise Encoding — inst[31:0]																														
Ext	**Type**	**Description**	**Explanation**	**Mnemonic**	**Operands**	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
I	**R - Type** "Register"	Set Less Than Unsigned	rd = (rs1 < rs2) ? 1 : 0 (unsigned)	**sltu**	rd, rs1, rs2	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**1**	**1**	rd					0	1	1	0	0	1	1
PSEUDO	**R - Type** "Register"	Set if < Zero	slt rd, rs, x0	**sltz**	rd, rs	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs					**0**	**1**	**0**	rd					0	1	1	0	0	1	1
PSEUDO	**R - Type** "Register"	Set if ≠ Zero	sltu rd, x0, rs	**snez**	rd, rs	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs					**0**	**0**	**0**	**0**	**0**	**0**	**1**	**1**	rd					0	1	1	0	0	1	1
I	**R - Type** "Register"	Shift Right Arithmetic	rd = rs1 >> rs2[4:0] (msb extends)	**sra**	rd, rs1, rs2	**0**	**1**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**1**	**0**	**1**	rd					0	1	1	0	0	1	1
I	**I - Type** "Immediate"	Shift Right Arithmetic Immediate	rd = rs1 >> shamt (msb extends) shamt	**srai**	rd, rs1, imm	**0**	**1**	**0**	**0**	**0**	**0**	**0**	shamt[4:0]					rs1					**1**	**0**	**1**	rd					0	0	1	0	0	1	1
						– 0 –																											4:0		
I	**R - Type** "Register"	Shift Right Logical	rd = rs1 >> rs2[4:0] (zero extends)	**srl**	rd, rs1, rs2	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**1**	**0**	**1**	rd					0	1	1	0	0	1	1
I	**I - Type** "Immediate"	Shift Right Logical Immediate	rd = rs1 >> shamt (zero extends) shamt	**srli**	rd, rs1, imm	**0**	**0**	**0**	**0**	**0**	**0**	**0**	shamt[4:0]					rs1					**1**	**0**	**1**	rd					0	0	1	0	0	1	1
						– 0 –																											4:0		
I	**R - Type** "Register"	Subtract	rd = rs1 - rs2	**sub**	rd, rs1, rs2	**0**	**1**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**0**	**0**	**0**	rd					0	1	1	0	0	1	1
I	**S - Type** "Store"	Store Word	rs2[31:0] → M[rs1 + imm] imm	**sw**	rs2, imm(rs1)	imm[11:5]							rs2					rs1					**0**	**1**	**0**	imm[4:0]					0	1	0	0	0	1	1
						– *11* –																				11:5										
																																	4:0		
PSEUDO	/	Store Global Word	auipc rt, imm[31:12] sw rs2, imm[11:0](rt)	**sw**	rs2, imm, rt	imm[31:12]																				rt					0	0	1	0	1	1	1
						imm[11:5]							rs2					rt					**0**	**1**	**0**	imm[4:0]					0	1	0	0	0	1	1
PSEUDO	/	Tail Call Far-Away Subroutine	auipc x6, imm[31:12] jalr x0, imm[11:0](x6)	**tail**	imm	imm[31:12]																				**0**	**0**	**1**	**1**	**0**	0	0	1	0	1	1	1
						imm[11:0]												**0**	**0**	**1**	**1**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	1	1	0	0	1	1	1
I	**R - Type** "Register"	Xor	rd = rs1 ^ rs2	**xor**	rd, rs1, rs2	**0**	**0**	**0**	**0**	**0**	**0**	**0**	rs2					rs1					**1**	**0**	**0**	rd					0	1	1	0	0	1	1
I	**I - Type** "Immediate"	Xor Immediate	rd = rs1 ^ imm imm	**xori**	rd, rs1, imm	imm[11:0]												rs1					**1**	**0**	**0**	rd					0	0	1	0	0	1	1
						– *11* –																				11:0										